

Machine Learning

4. *Supervised Learning 1*

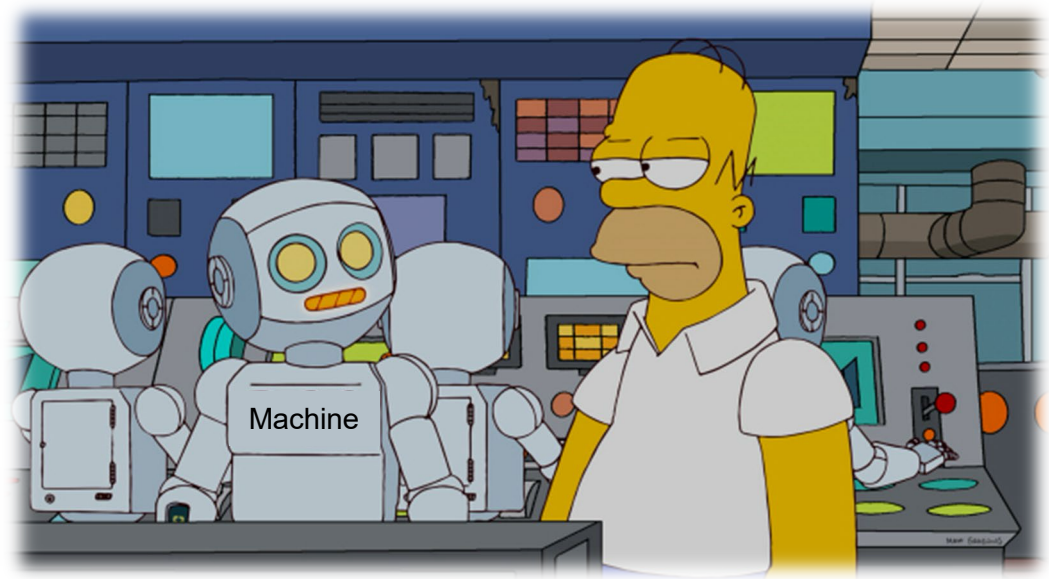
Sunwoo Lee
sunwool@inha.ac.kr



INHA UNIVERSITY

Today, we will learn...

- Supervised Learning 1
 - Linear Regression
 - Gradient Descent Algorithm
 - Logistic Regression



Do not share or distribute these slides!

저작권 안내 / Copyright Notice

본 사이트에서 수업 자료로 이용되는 저작물은 저작권법 제25조 수업목적저작물 이용 보상금제도의 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. **약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개·공유 및 수업 목적외의 사용을 금지합니다. (일부 공개로 제공되는 유튜브 강의를 불법으로 공유하거나 배포하는 행위를 금함)**

The teaching materials provided on this site are copyrighted and signed legally by the Korea Reproduction and Transmission Rights Association. Using these materials beyond the scope of the agreement is a violation against copyright laws. **Hence, one is not allowed to open or share the materials in public. Using them outside of the class is strictly prohibited.**

2020. 3. 11.

인하대학교·한국복제전송저작권협회

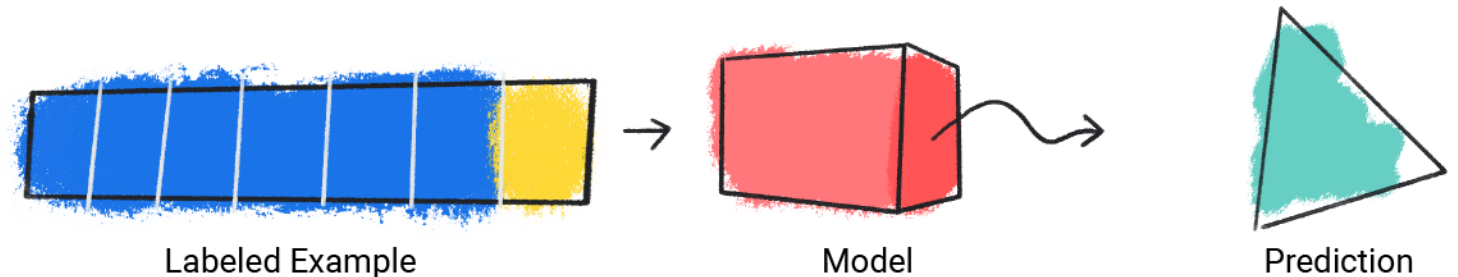
Inha University · Korea Reproduction and Transmission Rights Association

- ***Supervised Learning?***

- Linear Regression
- Logistic Regression

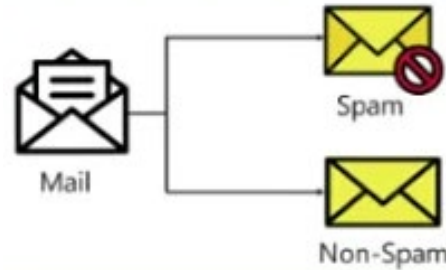
Supervised Learning

- The goal of ‘supervised learning’ is to make predictions from data.
- Core components:
 - Labeled Data
 - Model
 - Training
 - Evaluation (Validation)
 - Inference (Test)

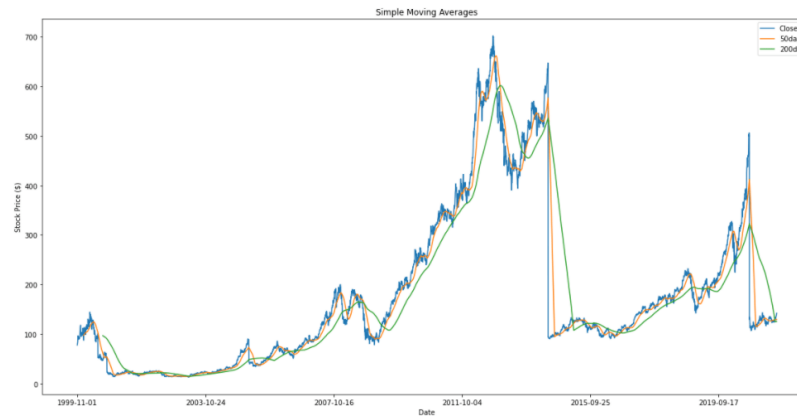


Types of Supervised Learning

- **Classification**: used for categorical response values, where the data can be separated into specific groups (classes).



- **Regression**: used for numerical continuous-response values.



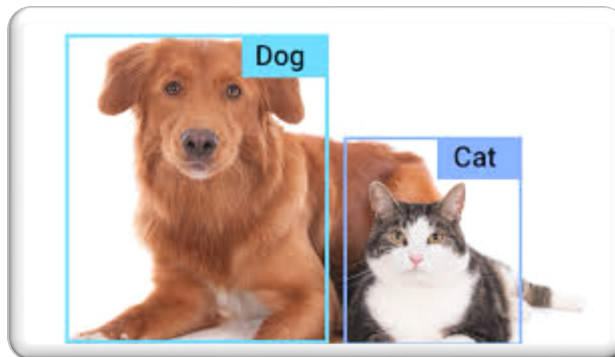
Formal Definition

- We would like to find **a mapping function f** : $X \rightarrow Y$ from a set of paired, labeled training data, $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, to predict the output y for unseen input x .
- $X \in \mathbb{R}^d$ can be multi-dimensional data.
- $Y \in \mathbb{R}^{d'}$ also can be any multi-dimensional data.
- We hope there exists an optimal solution f^* that accurately predict y for all existing possible x ! = 일반화
↳ ML의 핵심

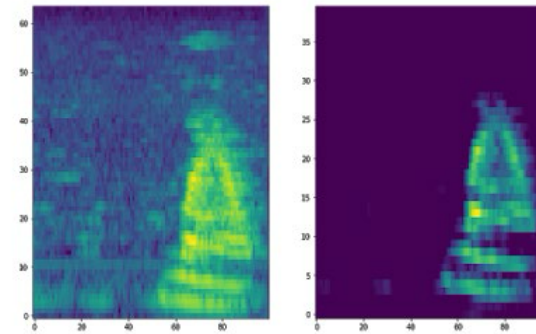
Supervised Learning in Real-World

- Many real-world problems can be formulated as a supervised learning problem.

Object Detection



Voice Classification

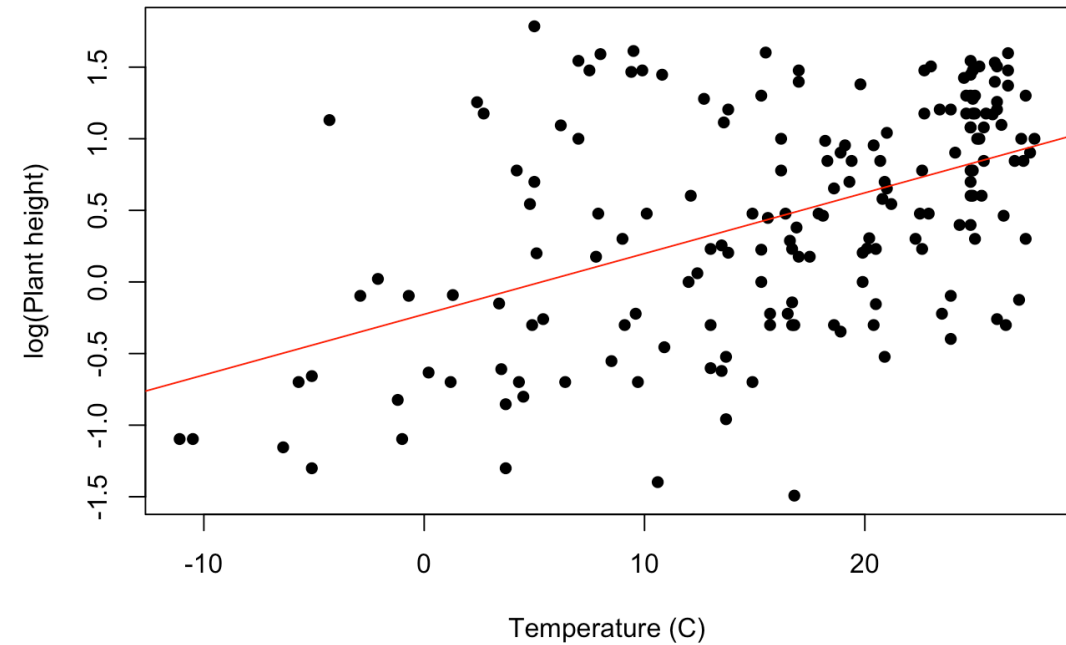


Left: Mel filterbanks for a person saying "wow". Right: LEAF's output for the same example, after training on a dataset of speech commands.

Weather Forecasting



- Supervised Learning?
- ***Linear Regression***
- Logistic Regression



Regression

- We consider data that consist of
 - Independent variable (observations) $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$, where $x_i \in \mathbb{R}^d$.
 - Dependent variable (labels) $\mathbf{y} = \{y_1, y_2, \dots, y_n\}$, where $y_i \in \mathbb{R}$.
- Our goal is to model the relationship between x and y .

$$f(x) = y' \approx y$$

Linear Regression

- A technique designed to model a **linear relationship** between independent (explanatory) data and dependent data.

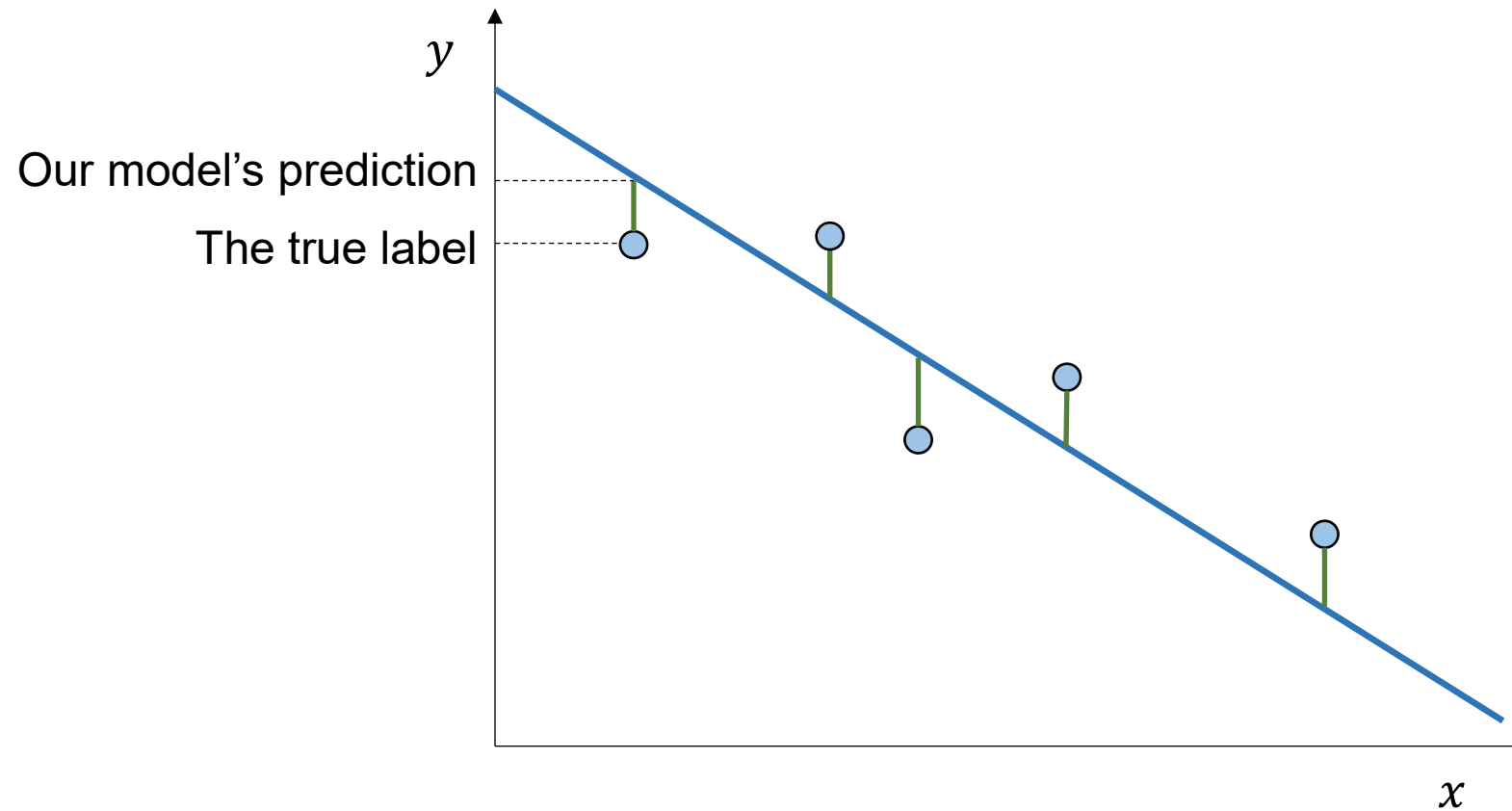
$$\begin{array}{c} f(x) = y' \\ \downarrow \\ ax + b = y' \end{array}$$

- The goal is to adjust the model parameters, a and b to make y' to be y , the label of x ,.

$$y = a_0 + a_1x_1 + \dots + a_dx_d = \sum_{i=1}^d a_ix_i + a_0$$

Multi-dimensional linear model

Schematic Illustration



Cost Function

- A function that measures ***how bad your model's prediction is.***
 - The quantified difference between the labels (true answers) and the model's outputs (predictions).
 - Also called 'loss' function.
- The higher the cost, the worse the model's quality.



Various Cost Functions

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Cross-Entropy
- Hinge Loss
- Kullback-Leibler Divergence (KL-Divergence)
- Much× 3 more customized loss functions

The problem solving begins with choosing an appropriate cost function!

Linear Regression with a Cost Function

- Our model makes a prediction y' based on the input data x .
- We already know that the correct label of x is y .
- Then, the error should be the ***difference*** between y' and y .

Mean Squared Error (MSE) cares of the **magnitude** of the error.

$$\begin{aligned} C(x) &= \frac{1}{2n} \sum_{i=1}^n \|y_i - y'_i\|^2 \\ &= \frac{1}{2n} \sum_{i=1}^n \|y_i - f(x)\|^2 \end{aligned}$$

↳ 모든 값에서 마분가능

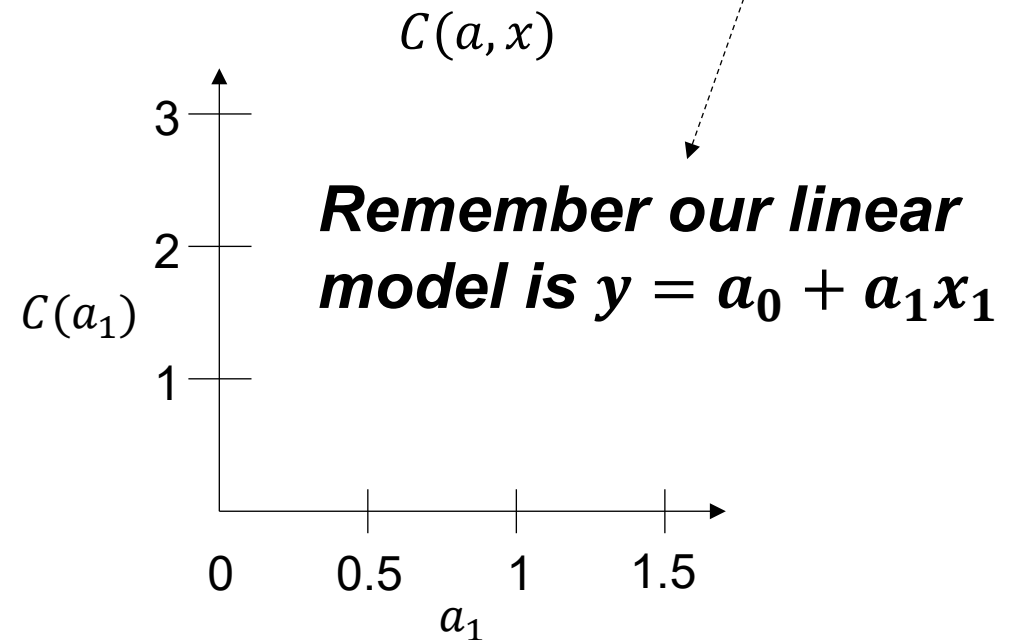
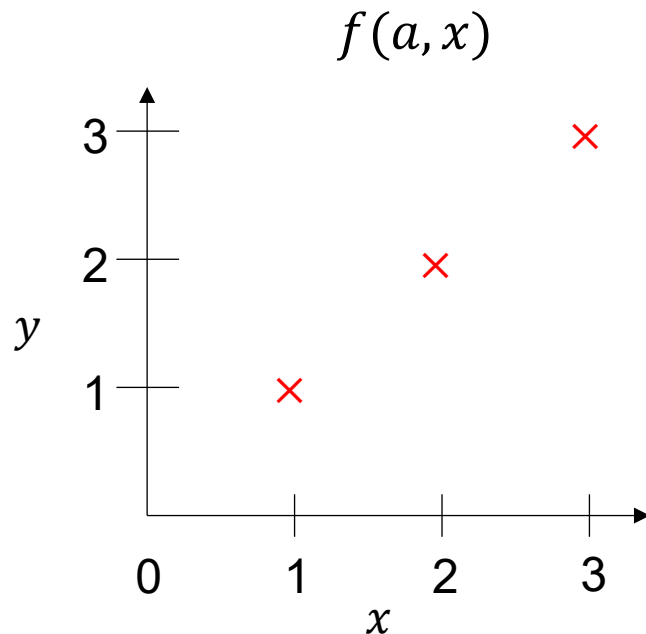
Mean Absolute Error (MAE) focuses on the **sign** of the error.

$$\begin{aligned} C(x) &= \frac{1}{n} \sum_{i=1}^n |y_i - y'_i| \\ &= \frac{1}{n} \sum_{i=1}^n |y_i - f(x)| \end{aligned}$$

↳ 0 이서 마분 불가능, 그래도 0 이리 자평이선 가능하므은 사용함

Intuition Behind Cost Function (1/5)

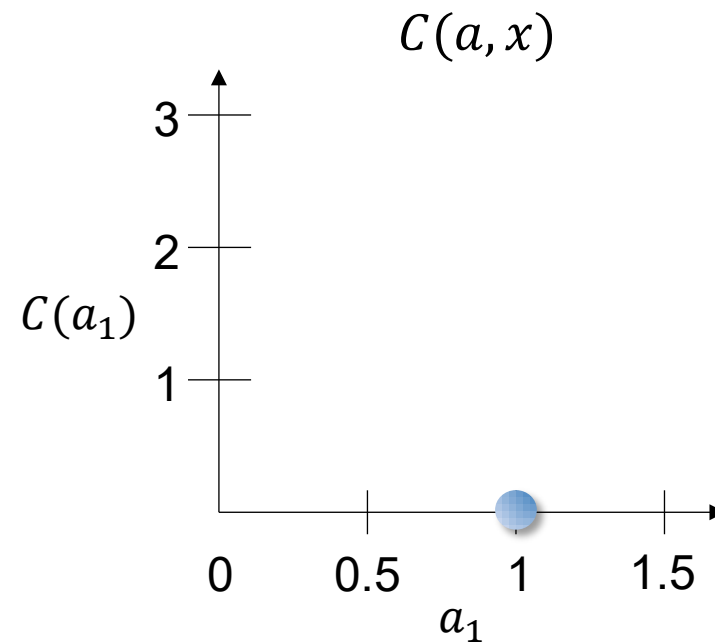
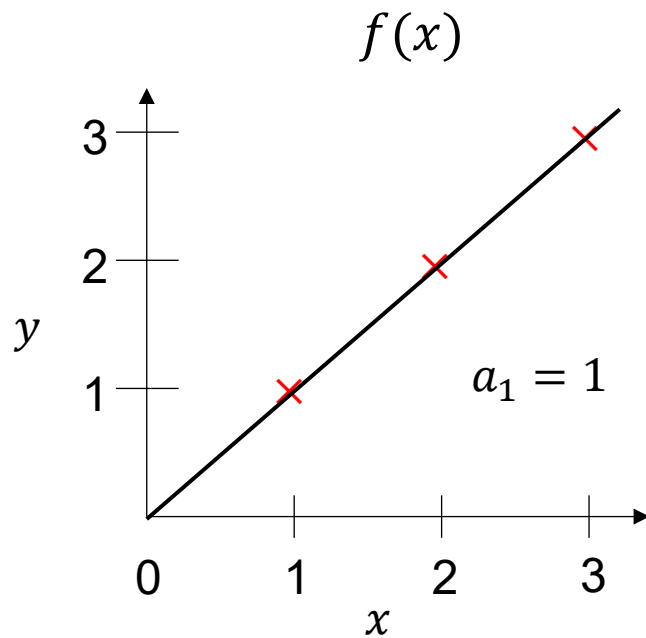
- Let us consider MSE: $C(\mathbf{a}, \mathbf{x}) = \frac{1}{2n} \sum_{i=1}^n \|y_i - y'_i\|^2 = \frac{1}{2n} \sum_{i=1}^n \|y_i - f(\mathbf{a}, x_i)\|^2$.
- Say $x \in \mathbb{R}$ so $\mathbf{a} = [a_0, a_1]$.
- We fix a_0 to 0 for convenience.
- We are given with three data points: (1,1), (2,2), and (3,3).



Intuition Behind Cost Function (2/5)

- If $a_1 = 1$, then, $f(x) = x$.

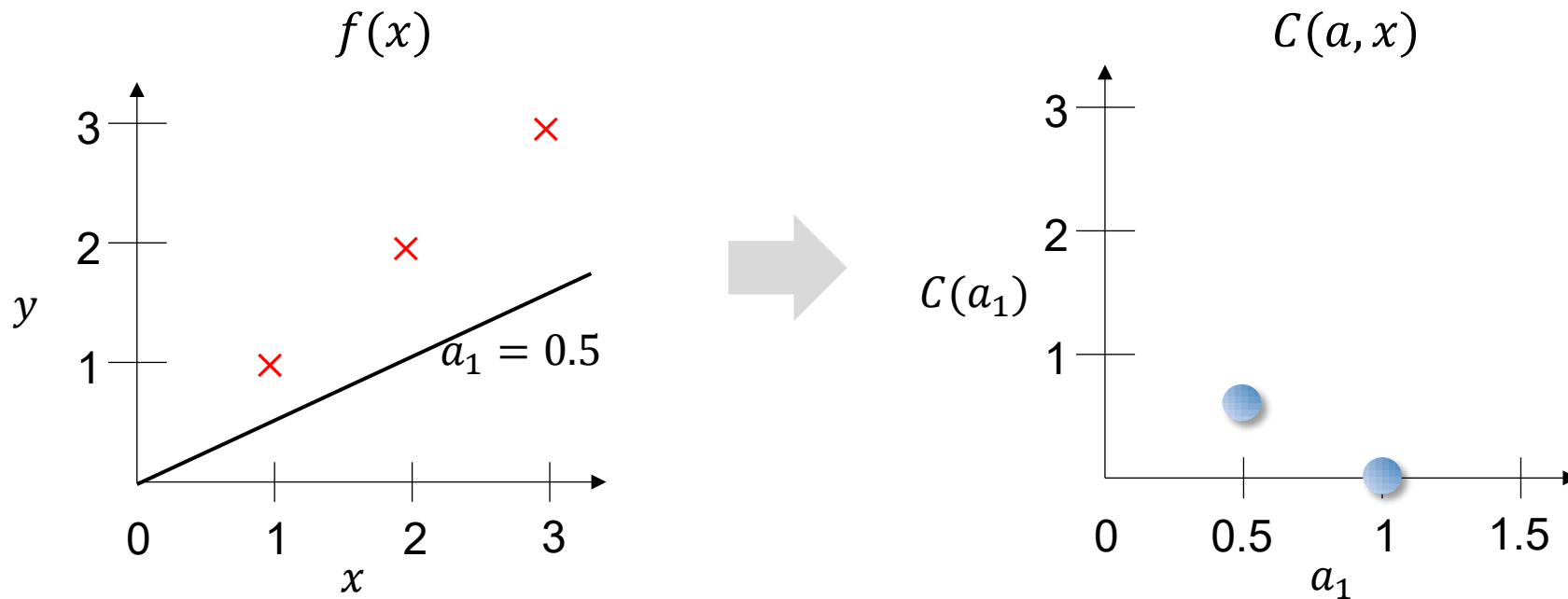
$$C(x) = \frac{1}{2 \times 3} ((1 - 1)^2 + (2 - 2)^2 + (3 - 3)^2) = 0$$



Intuition Behind Cost Function (3/5)

- If $a_1 = 0.5$, then, $f(x) = 0.5x$.

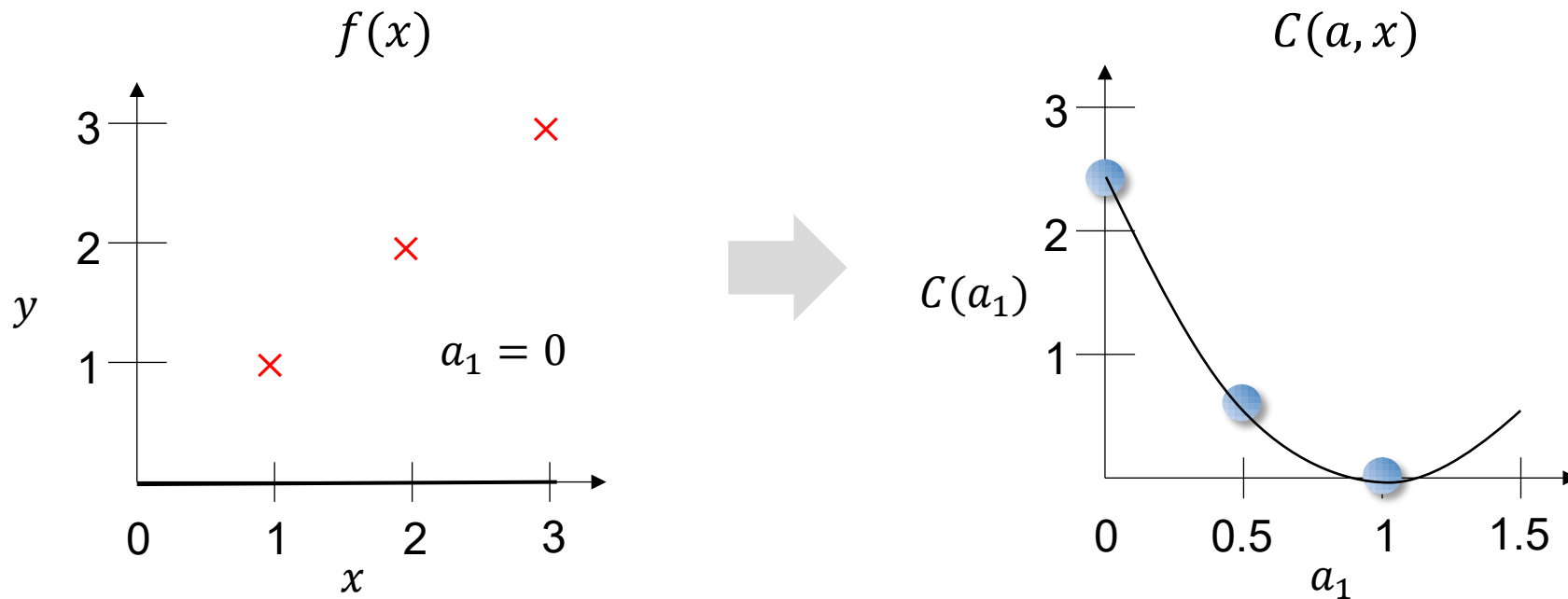
$$C(x) = \frac{1}{2 \times 3} ((0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2) \approx 0.58$$



Intuition Behind Cost Function (4/5)

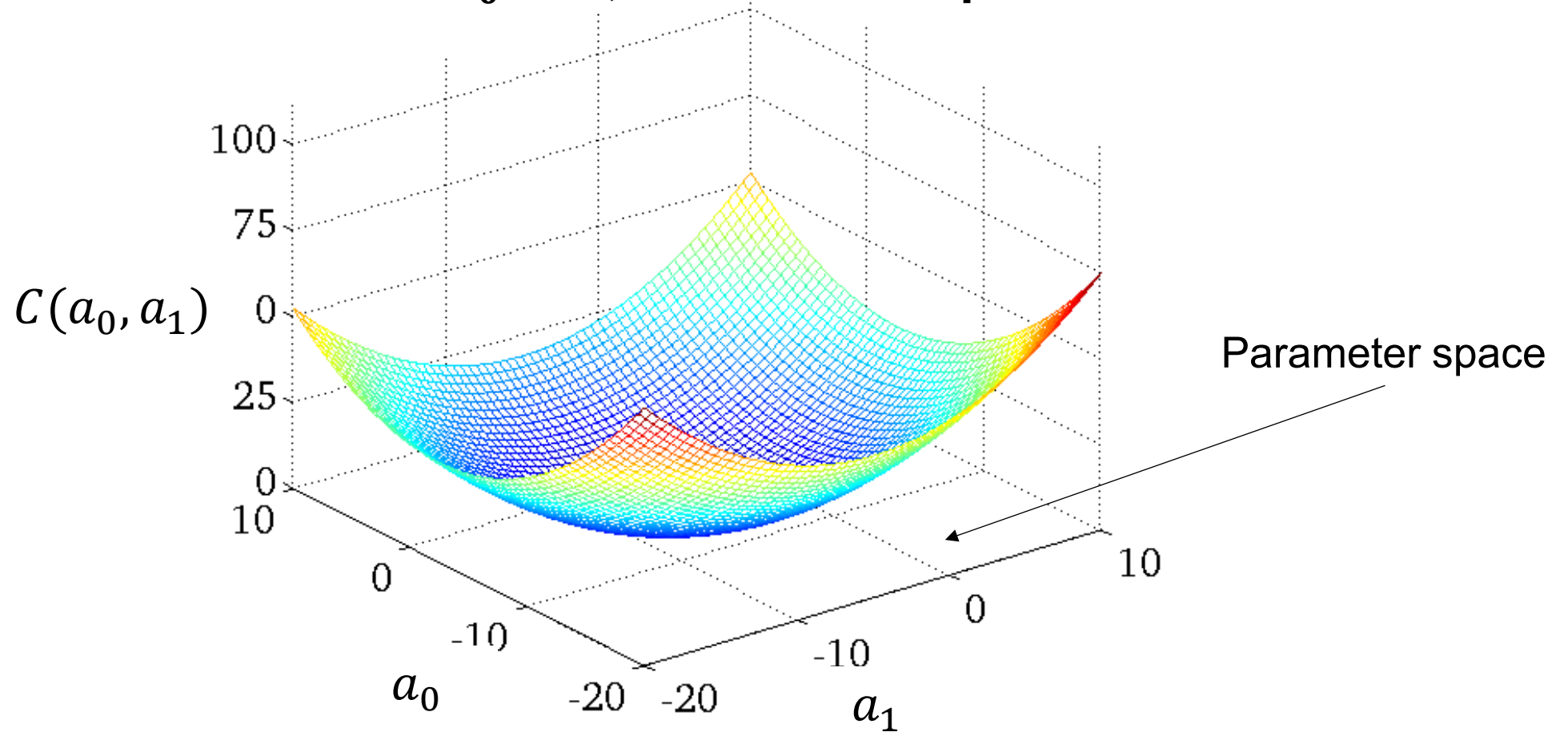
- If $a_1 = 0$, then, $f(x) = 0$. → $C(x)$ 가 2차식, 따라서 볼록함수이고 미분가능하며 Global Optimum이 유일하게 존재함

$$C(x) = \frac{1}{2 \times 3} ((0 - 1)^2 + (0 - 2)^2 + (0 - 3)^2) \approx 2.33$$

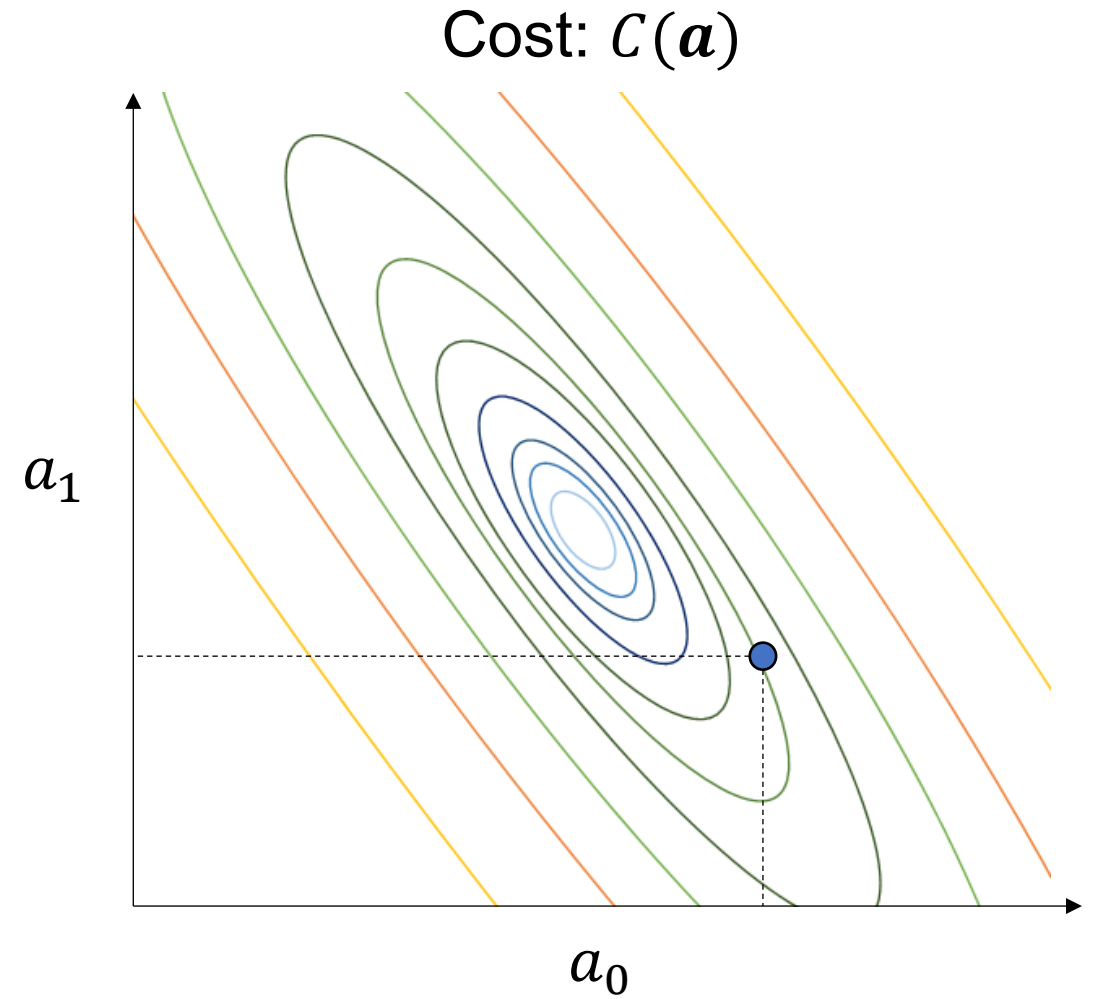
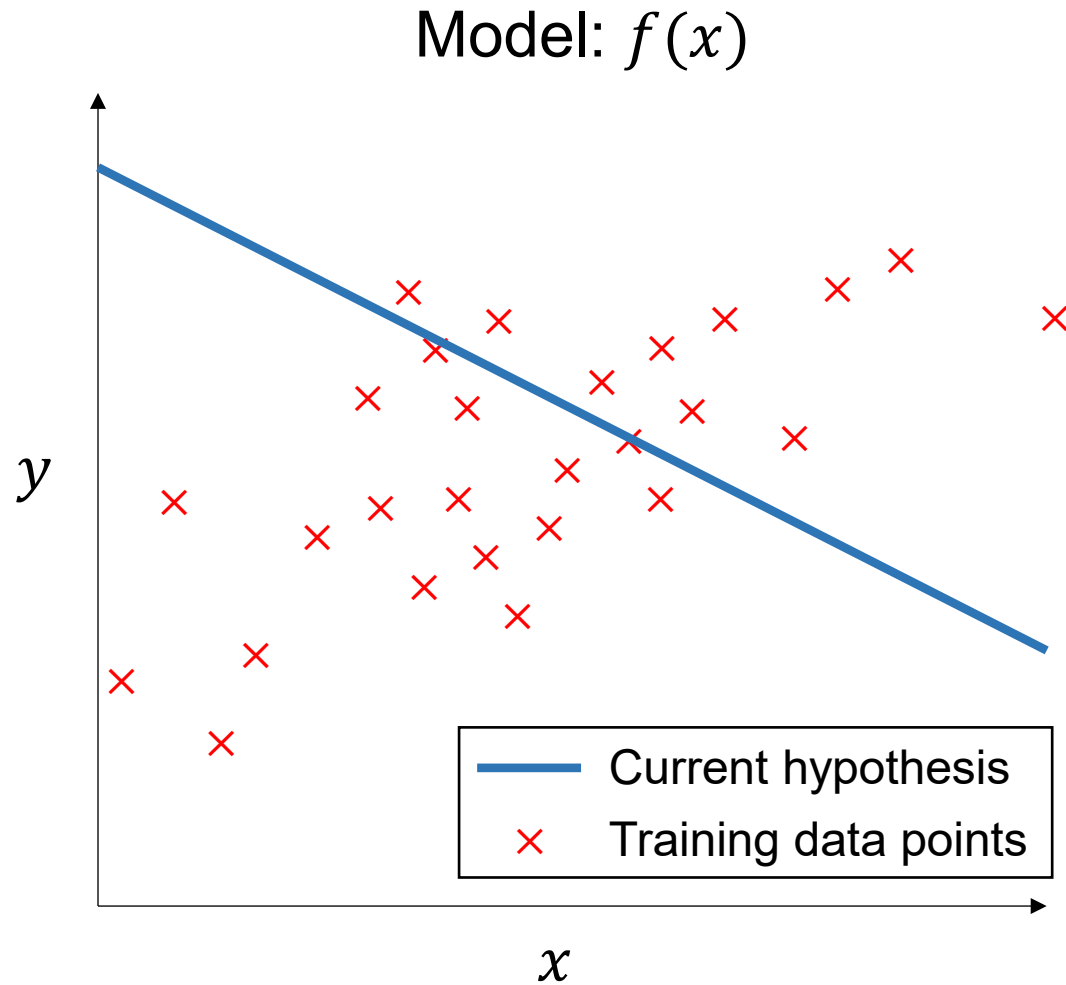


Intuition Behind Cost Function (5/5)

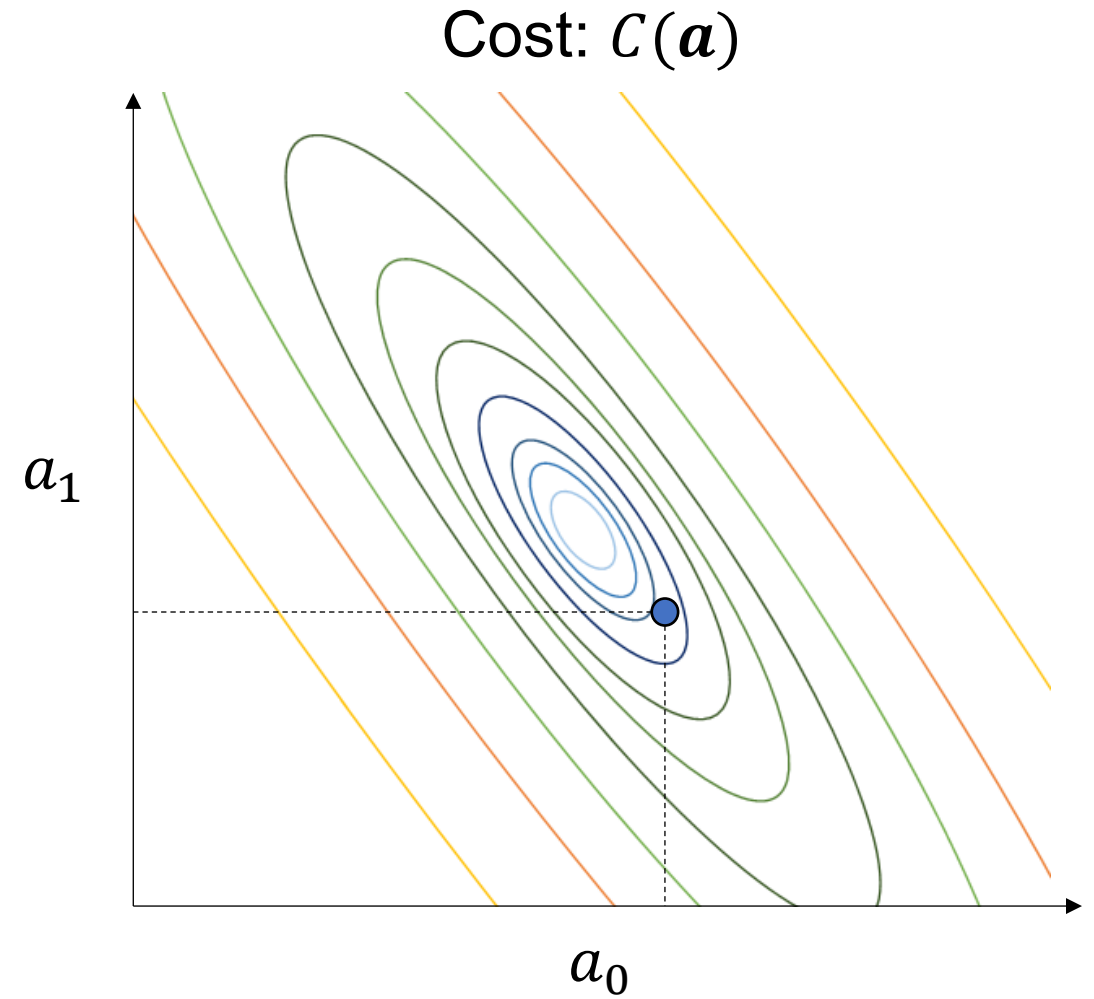
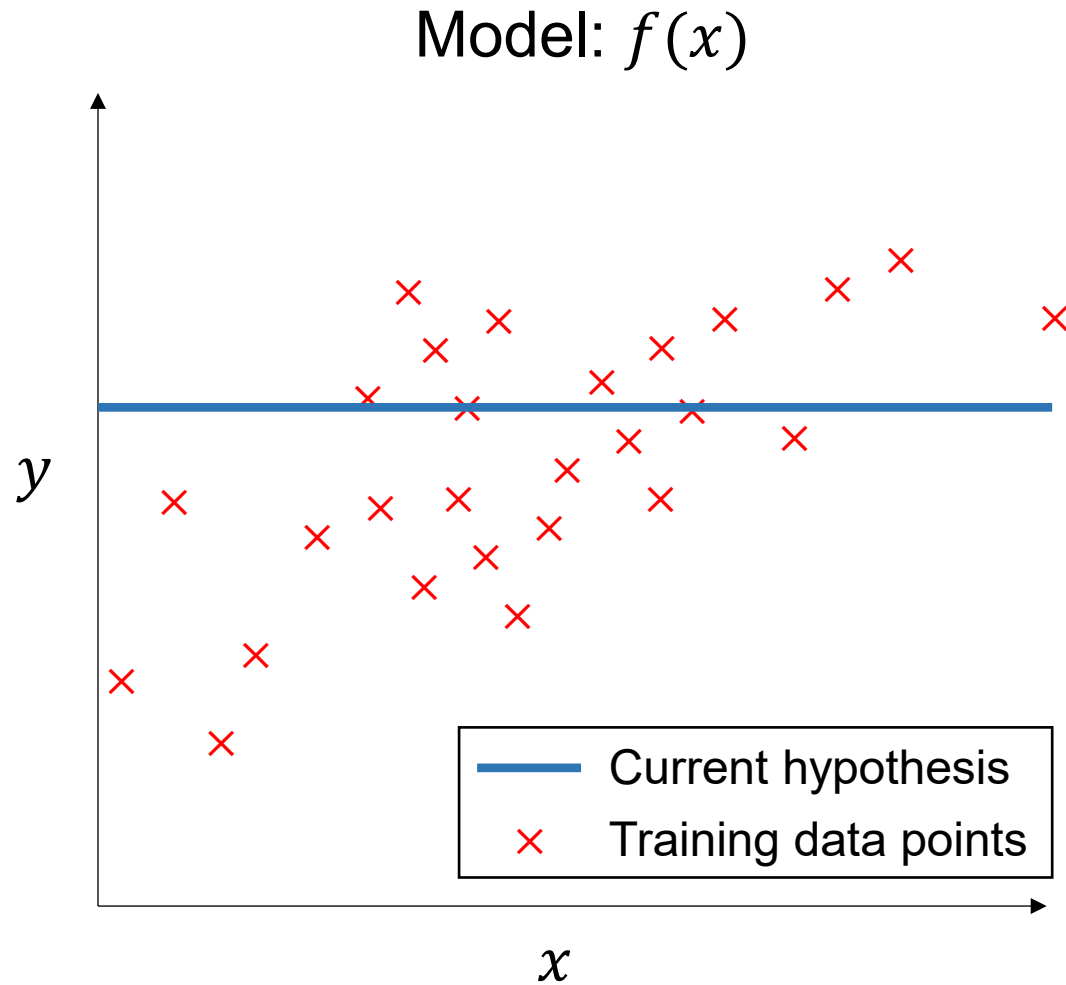
If we do not fix a_0 to 0, it becomes a paraboloid.



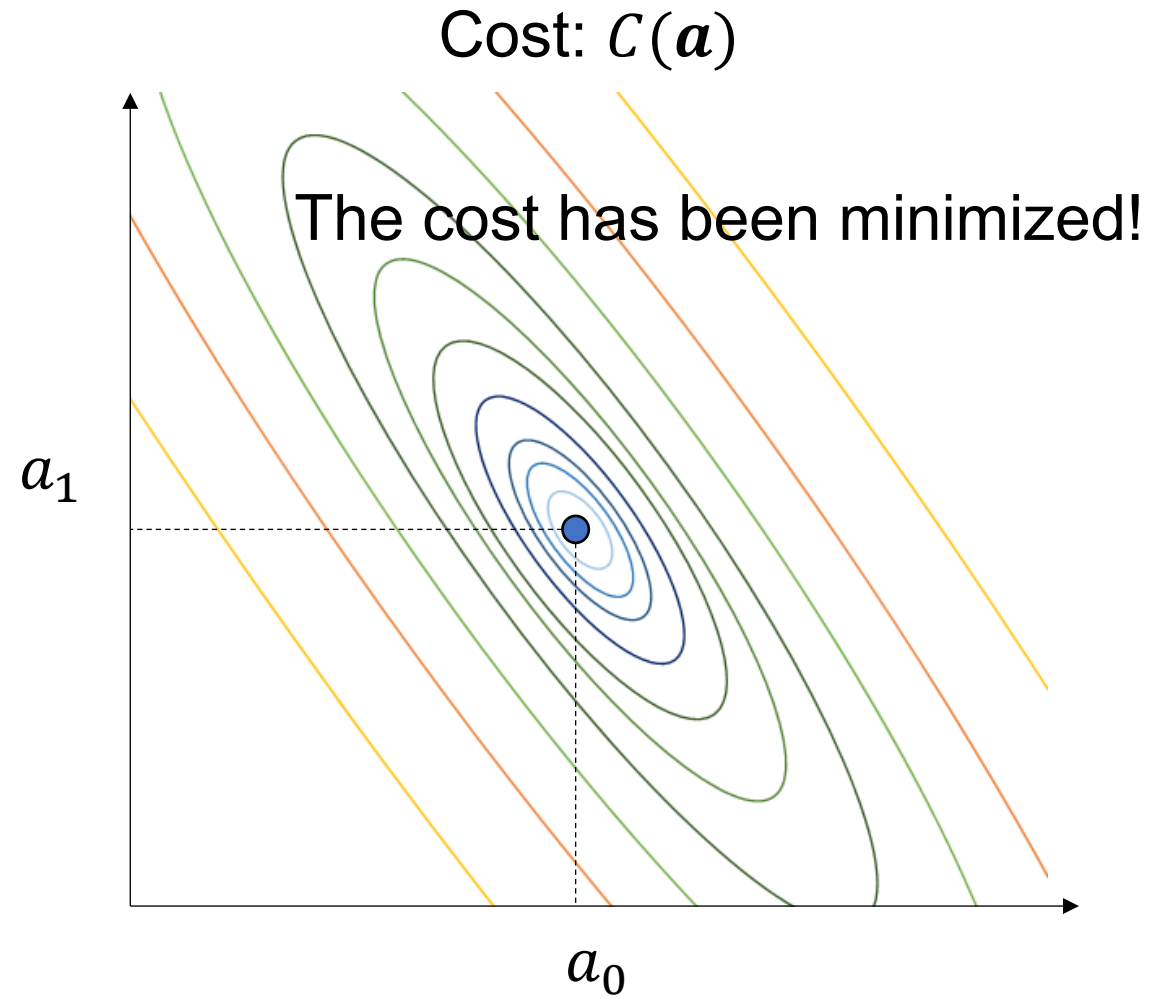
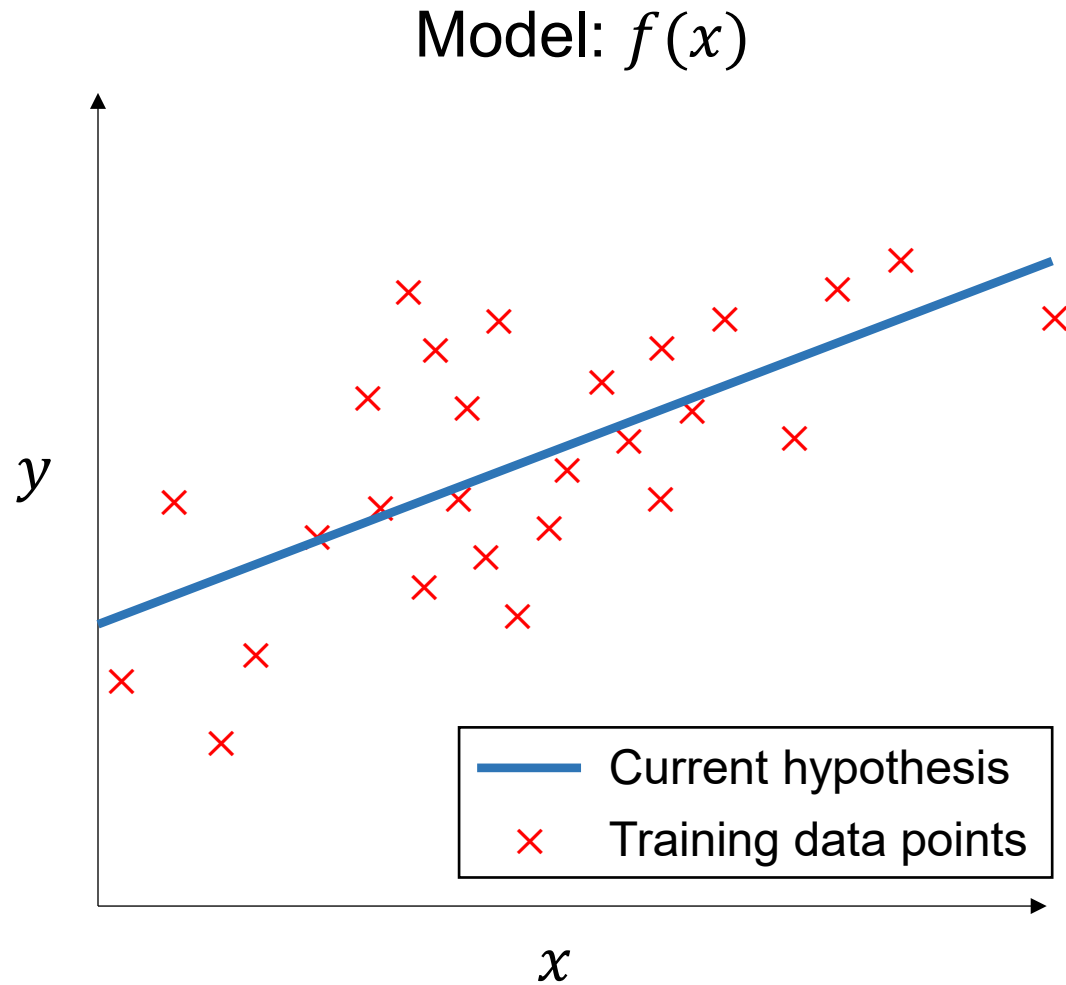
What Happens When Adjusting Our Model (1/3)



What Happens When Adjusting Our Model (2/3)



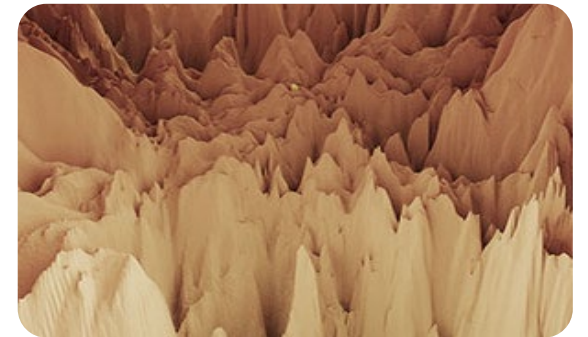
What Happens When Adjusting Our Model (3/3)



Training

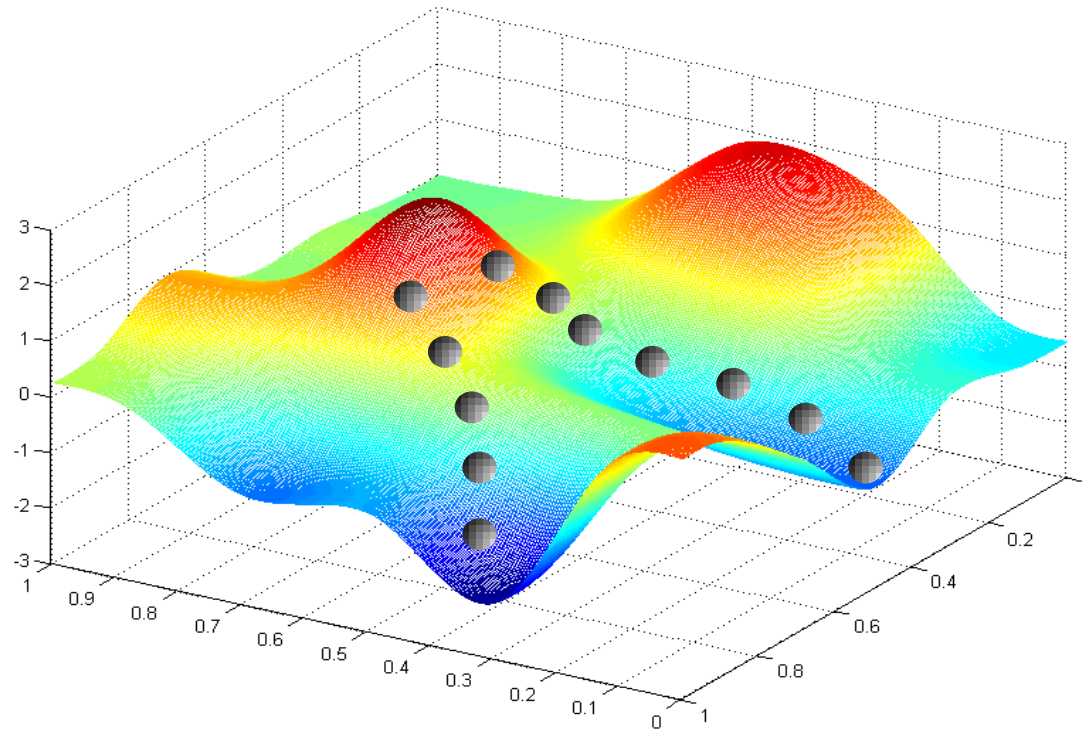
- In the previous example, we adjusted the model parameters, a_0 and a_1 , to minimize the cost value: $C(\mathbf{a}, \mathbf{x})$.
- This procedure is called '**training**'.
- In usual, the parameter space is very $\times 3$ large.
 - E.g., GPT-3 model has ~175 billion parameters. That is, the number of all possible combinations of the parameter values is near **infinite**.
- How would you find the best model parameters?

The actual loss landscape of CIFAR-10.



Parameter Search

- Start from a random initial point and repeatedly adjust the model parameters towards a better region!



The initial point is important!

Training Algorithms

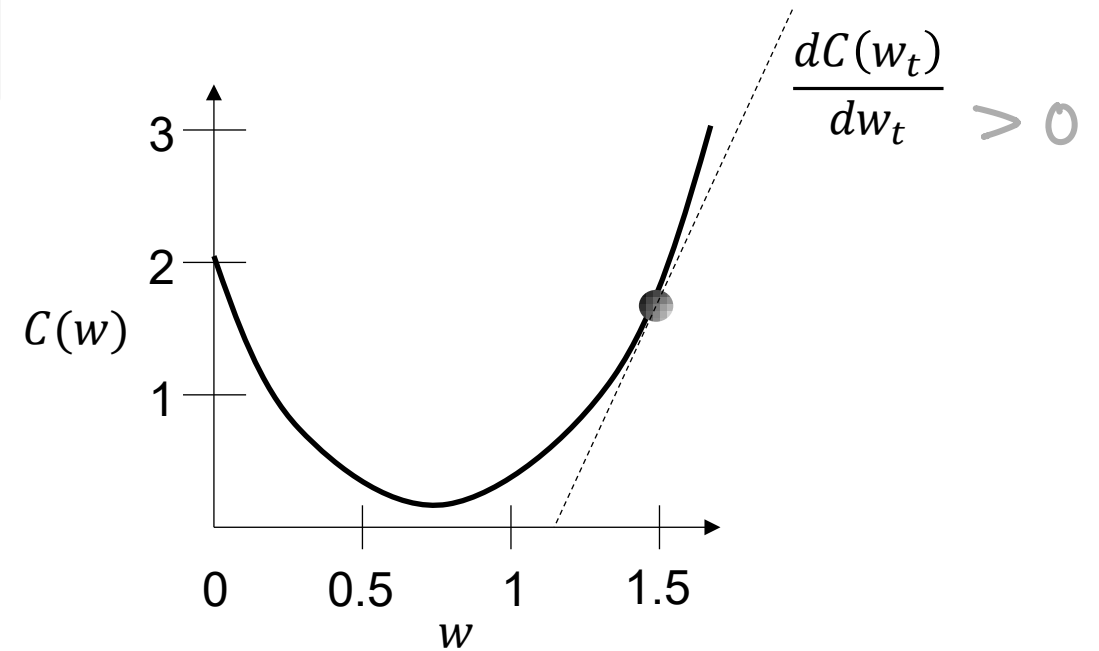
- Numerical optimization algorithms solve this minimization problem efficiently!
- Optimization (minimization) problems
 - Given a function $f: A \rightarrow \mathbb{R}$, find an element $x' \in A$ such that $f(x') \leq f(x)$ for all $x \in A$.
- Algorithms
 - First-order methods: GD, SGD, Momentum, AdaGrad, Adam, etc.
 - Second-order methods: Newton's methods, Quasi-Newton (BFGS), etc.

Gradient Descent

Algorithm: Gradient Descent (GD)

1. Initialize the model w
2. While **termination condition** is not met:
3. $w_{t+1} = w_t - \eta \frac{dC(w_t)}{dw_t}$

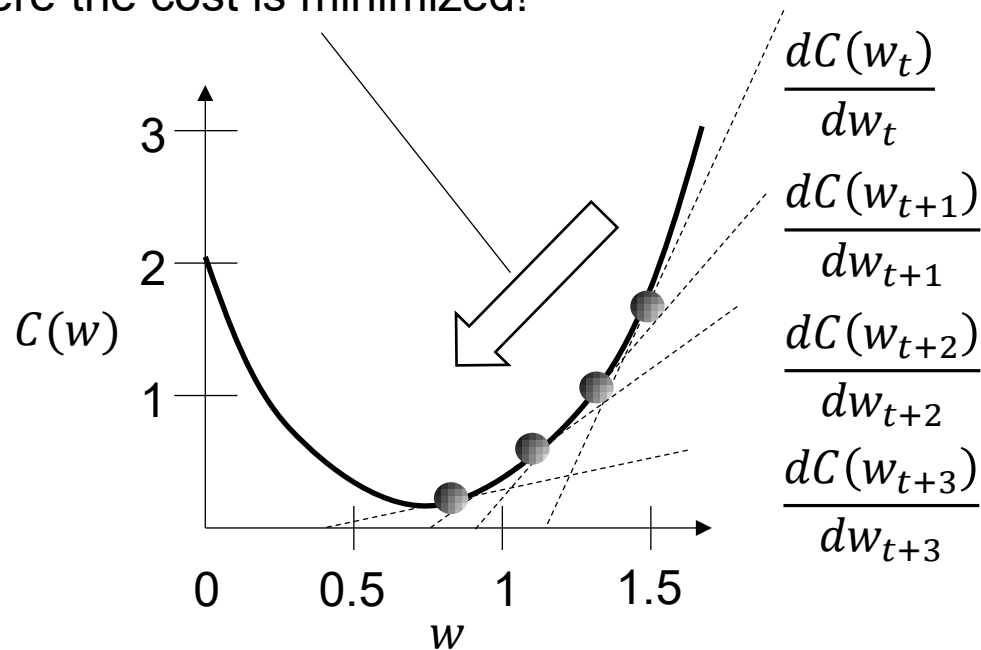
Learning rate (a small constant)



Gradient Descent

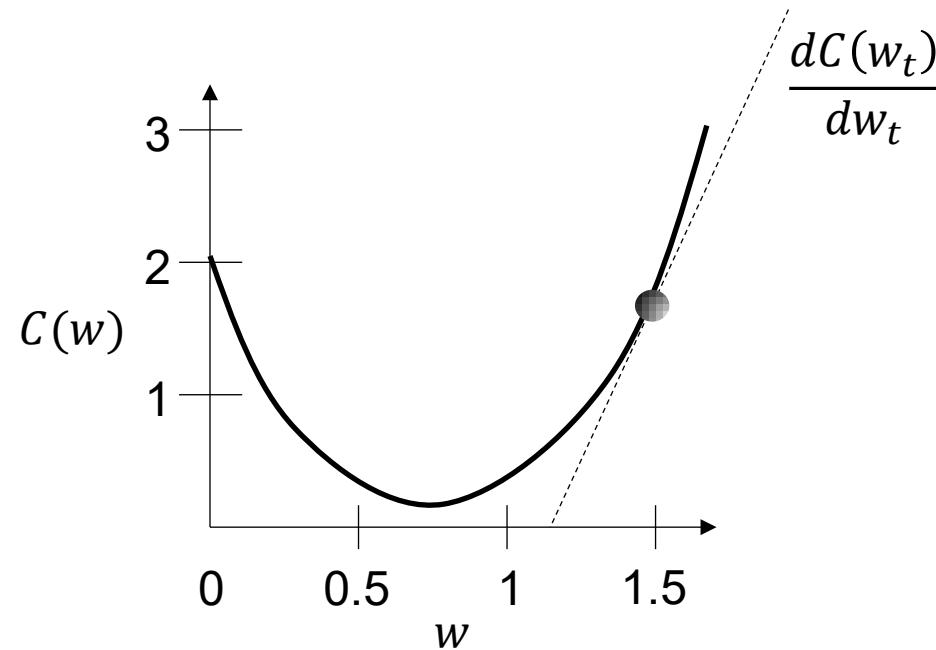
- The repeated updates lead the model to the minimum.

The model moves towards the point where the cost is minimized!



Gradient Descent

- Nice properties of gradients:
 - The **sign** of the gradients shows us which direction to go. 이동하는 방향을 결정
 - The **magnitude** of the gradients tends to decrease as the training progresses so that we can 'finish' the training. 이동하는 거리를 결정



Gradient Descent

- Gradient Descent has been widely used in many applications including machine learning, signal processing, statistics, etc.
- It always finds ***the optimal gradients***, and thus it ***guarantees the convergence*** of the loss function.

↓
모델의 파라미터, 손실값, 예측 성능이 어떤 안정된 상태에 가까워져서, 반복을 더 해도 큰 변화가 잘 일어나지 않는 상태
!= 전역 최적해를 찾음

- We will get back to here at the end of the semester (optimization and deep learning).

Solving Linear Regression Problems with GD

$$\begin{aligned}
 \frac{dC(w)}{dw_j} &= \frac{d}{dw_j} \left(\frac{1}{2n} \sum_{i=1}^n \left(\underline{f(w, x^i)} - y^i \right)^2 \right) \\
 &= \frac{d}{dw_j} \left(\frac{1}{2n} \sum_{i=1}^n \left(\underline{\sum_{j=1}^d w_j x_j^i + w_0} - y^i \right)^2 \right) \\
 &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{k=1}^d w_k x_k^i + w_0 - y^i \right) \times \underline{\frac{d}{dw_j} \left(\sum_{k=1}^d w_k x_k^i + w_0 - y^i \right)} \\
 &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{k=1}^d w_k x_k^i + w_0 - y^i \right) \underline{x_j^i} \\
 &= \frac{1}{n} \sum_{i=1}^n \underbrace{\left(f(w, x^i) - y^i \right)}_{\text{Error}} \underbrace{x_j^i}_{\text{Input data}}
 \end{aligned}$$

Solving Linear Regression Problems with GD (2/2)

- Parameter update rule of GD:

- $w_{t+1} = w_t - \eta \frac{dC(w_t)}{dw_t}$

- We now know $\frac{dC(w_t)}{dw_t}$ for every model parameter!

$$\frac{1}{n} \sum_{i=1}^n (f(w, x^i) - y^i) x_j^i$$

Calculate error of all individual data.

n 개

This is the input data.

$n \times d$ 행렬

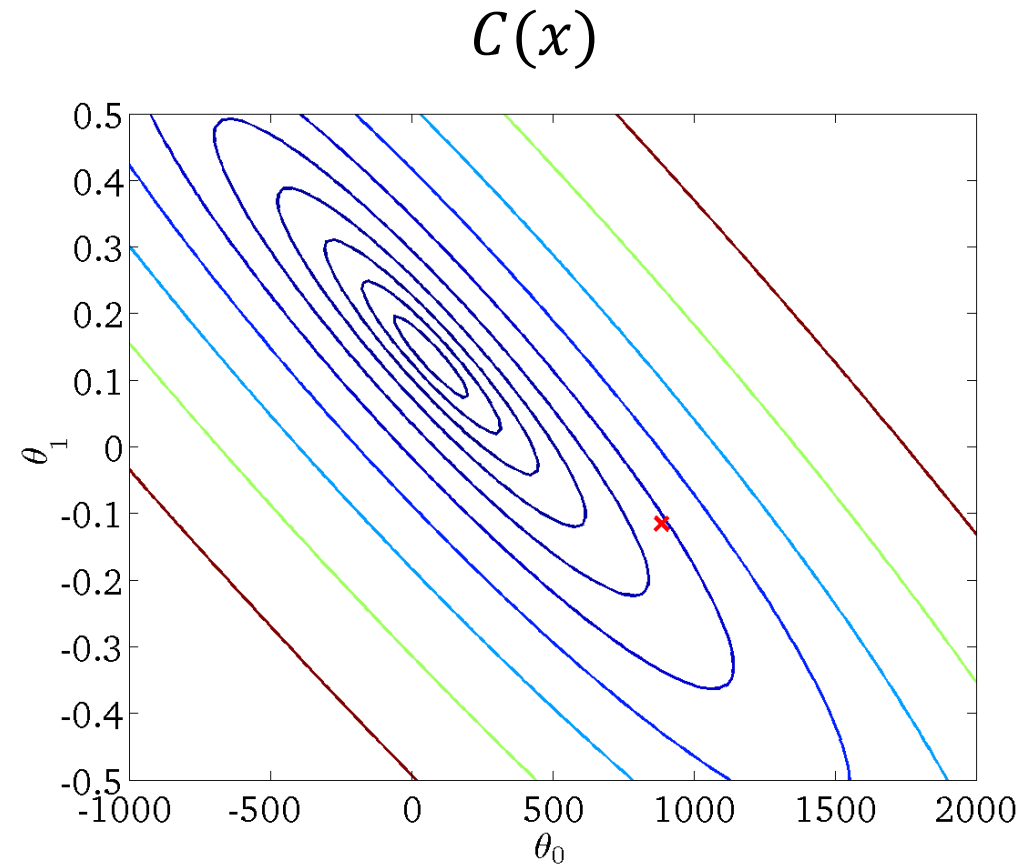
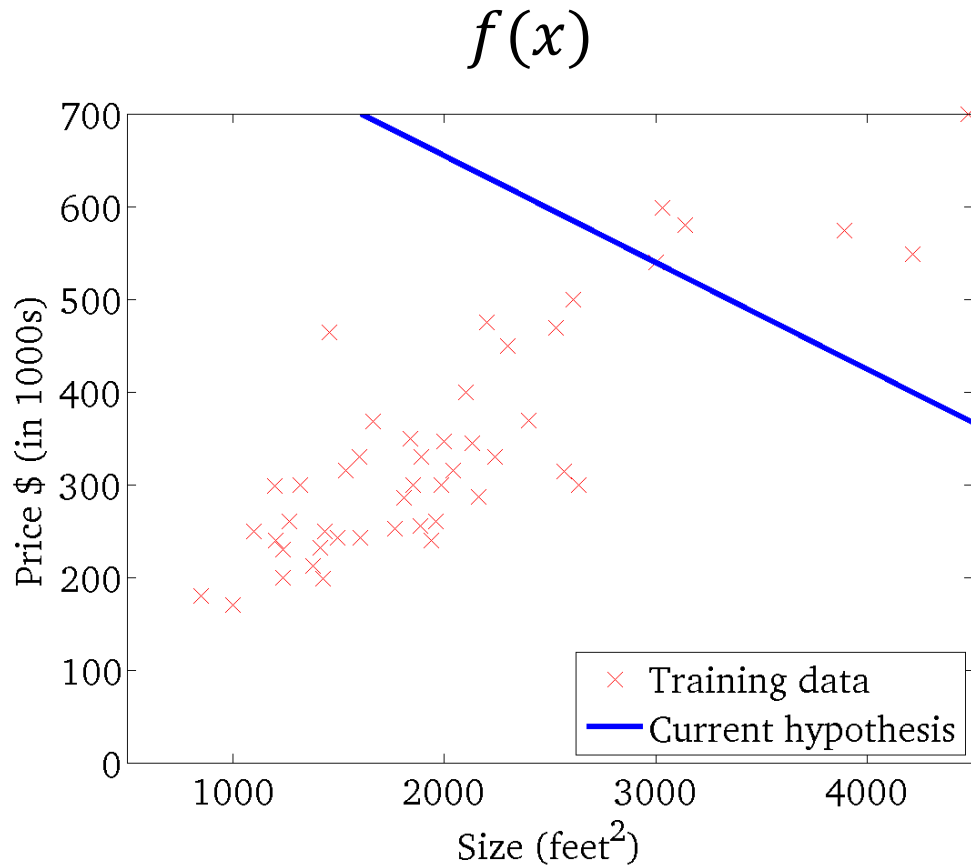
Solving Linear Regression Problems with GD

- Repeat calculating the gradient of all individual parameters and updating them until the following 'termination condition' is met.

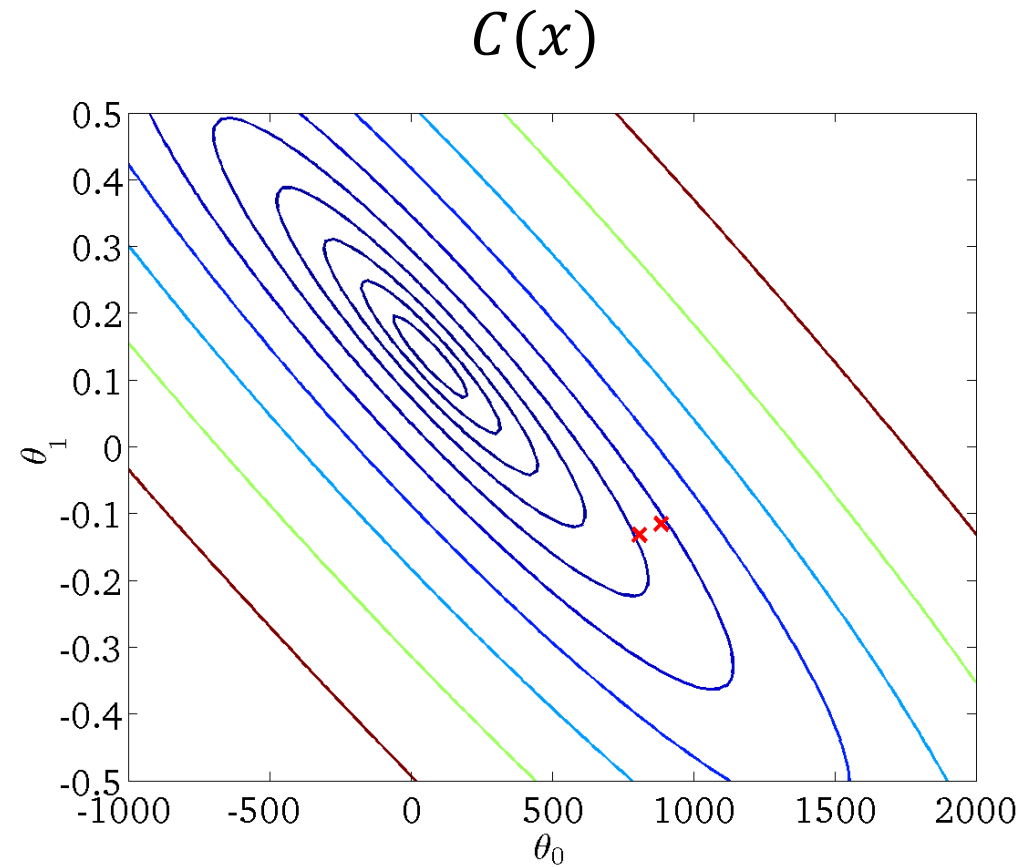
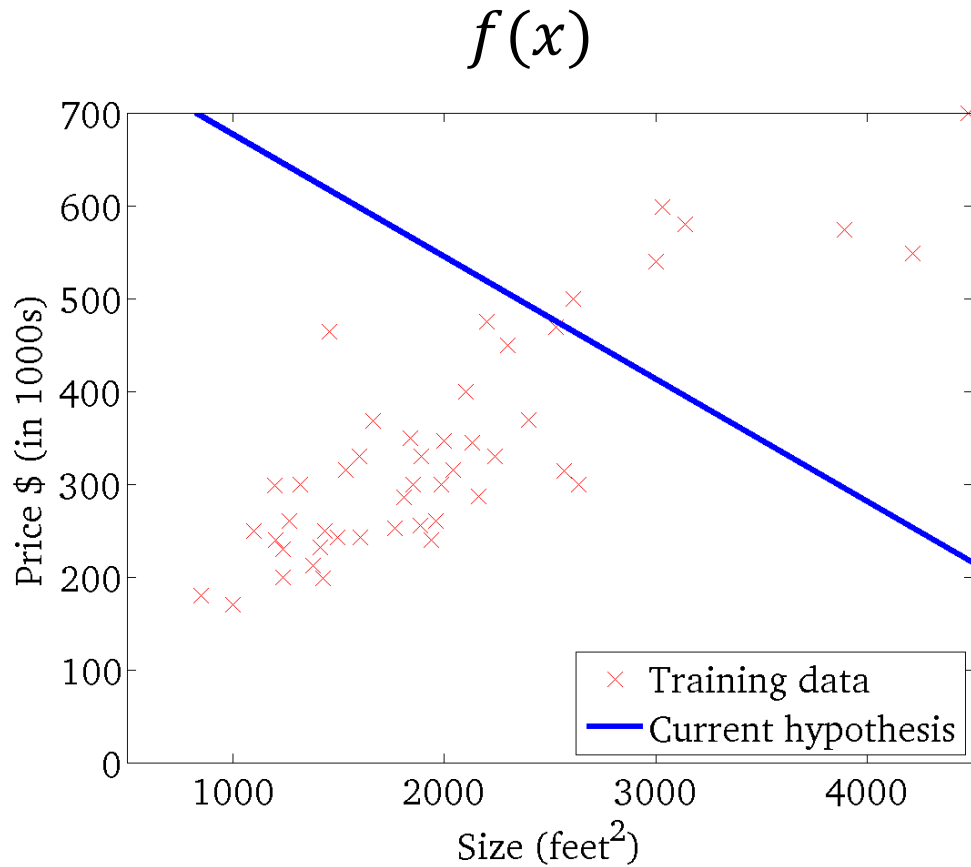
$$\|w_{new} - w_{old}\|_2 \leq \epsilon$$

- In theory, ϵ can be zero, but in practice, ϵ is usually a small constant.

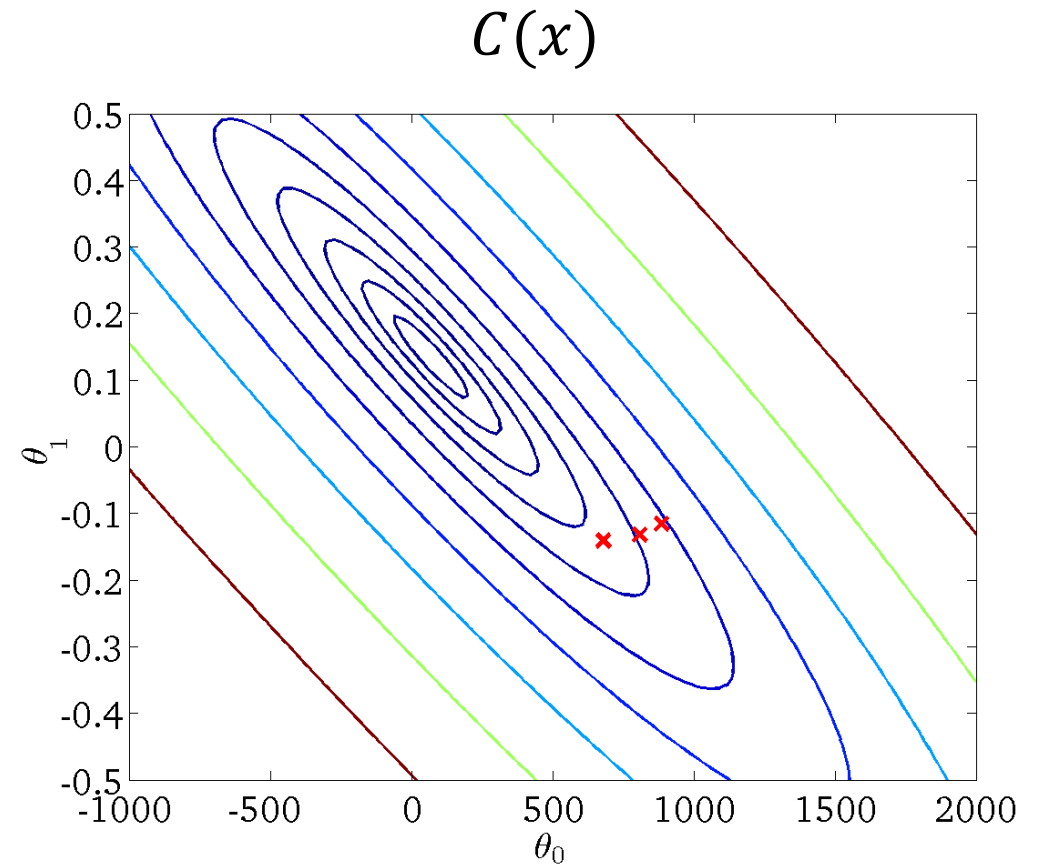
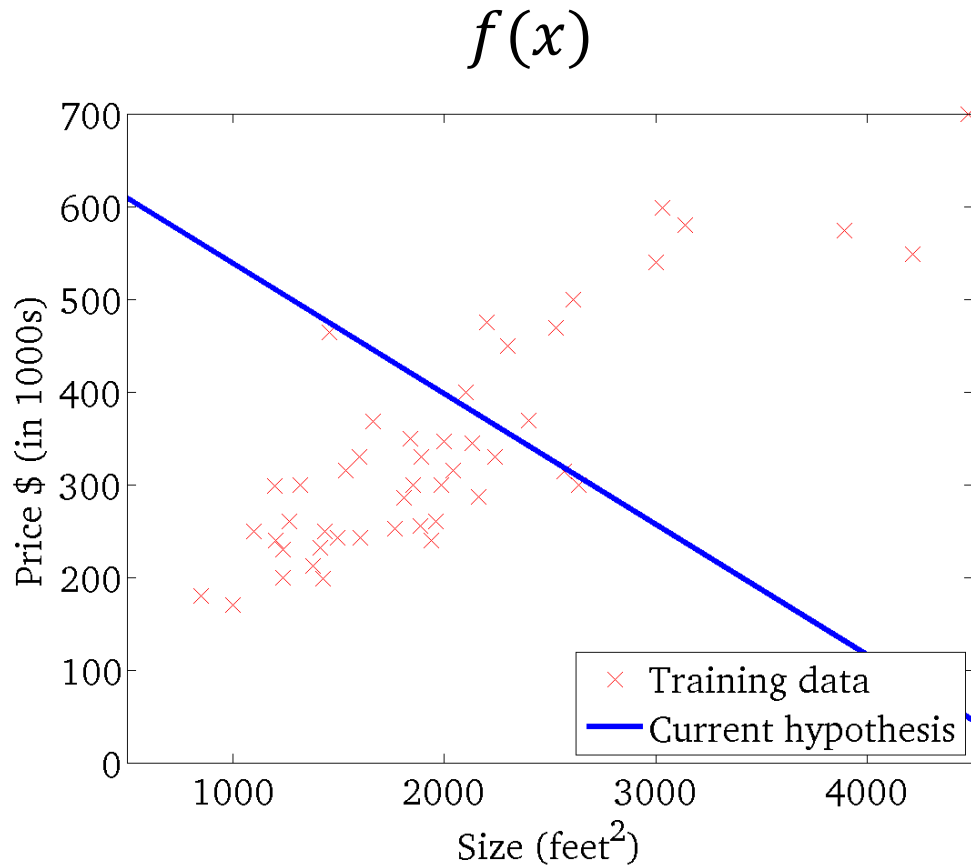
Training Procedure Example



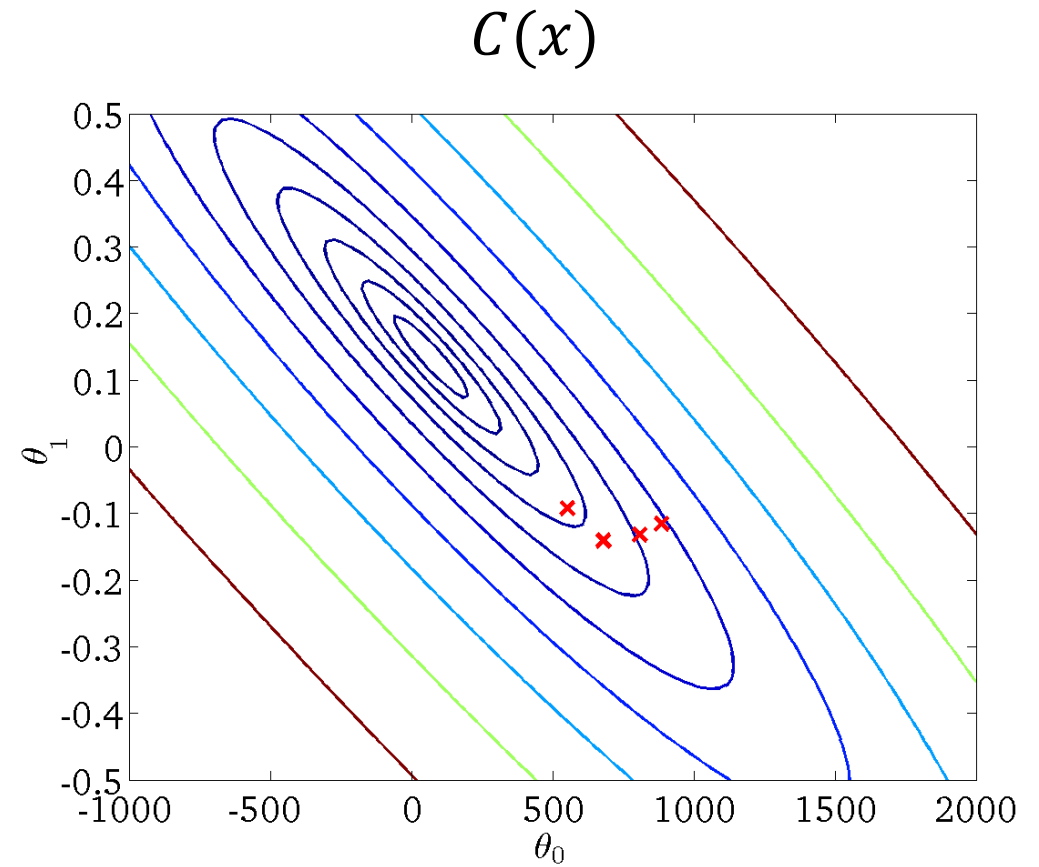
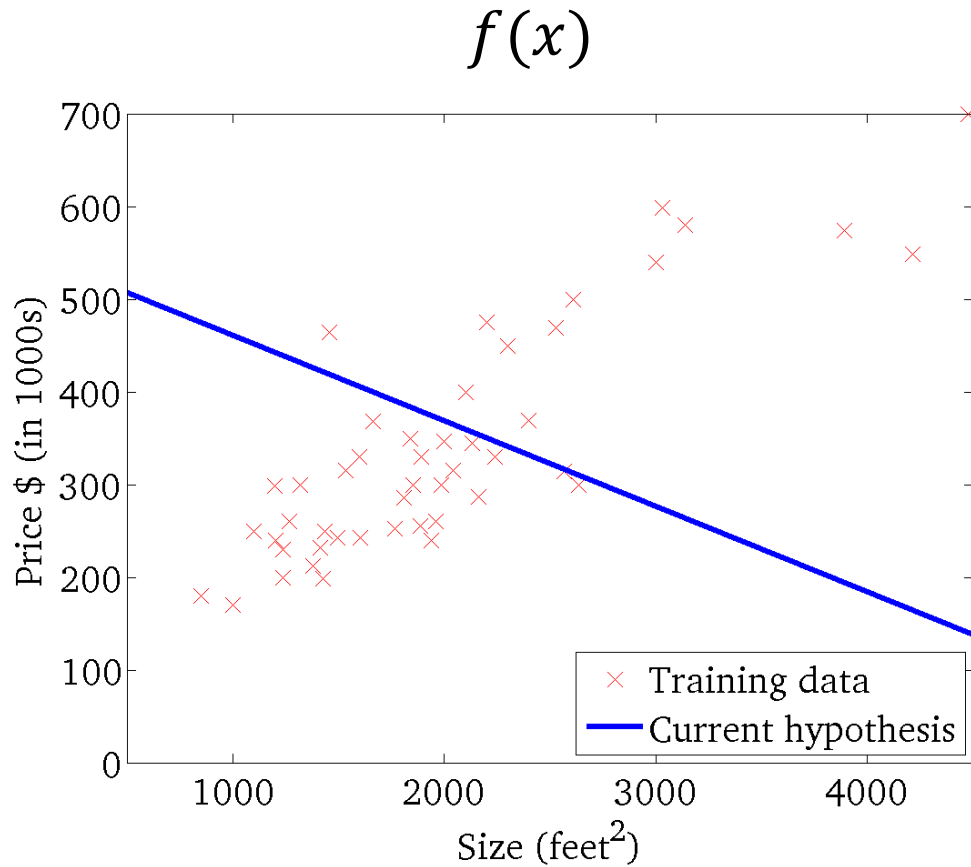
Training Procedure Example



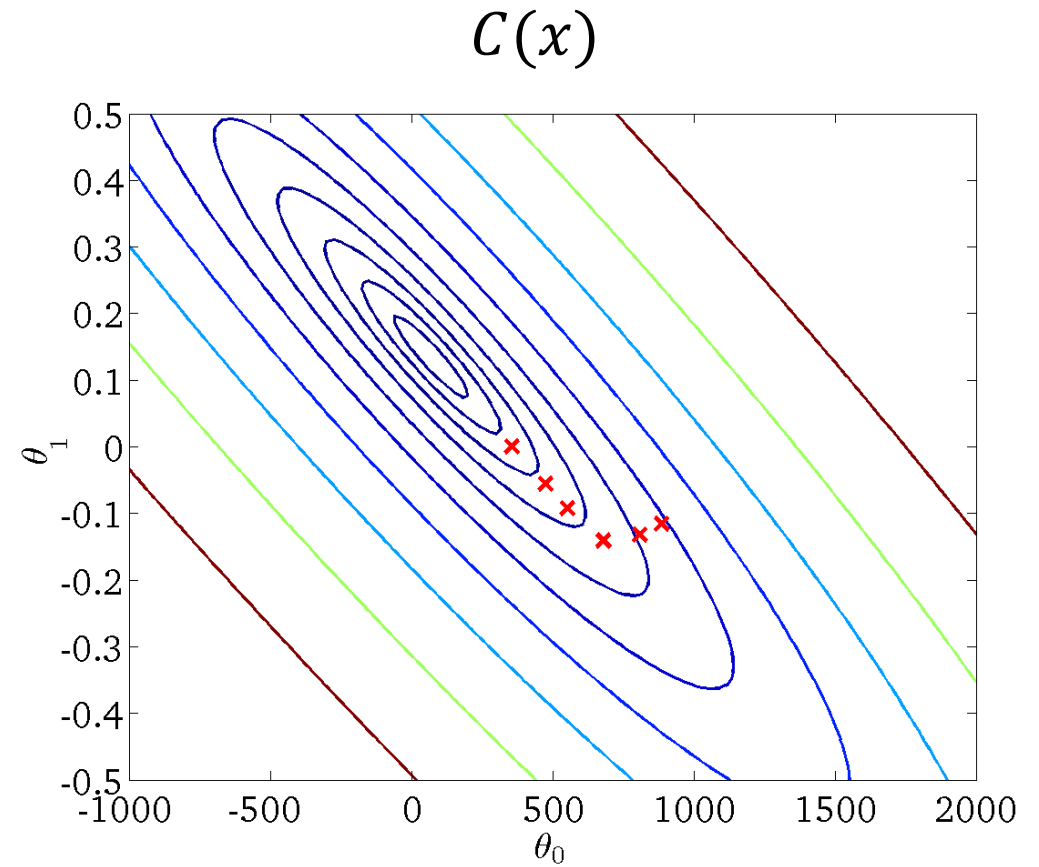
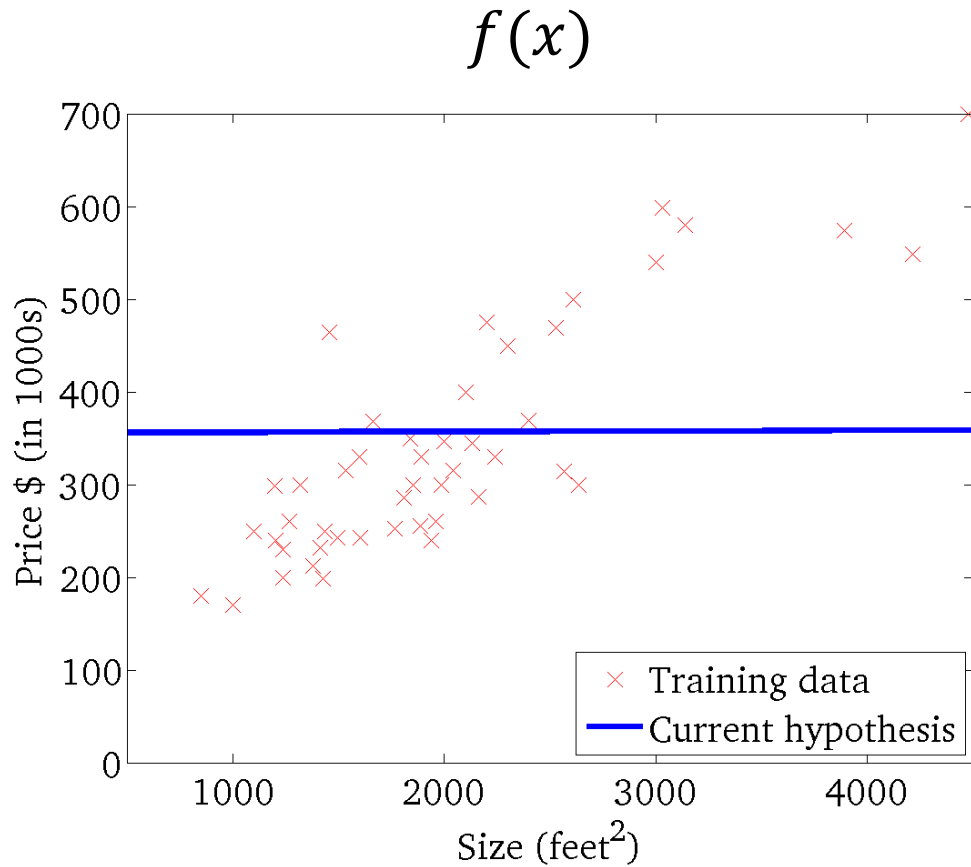
Training Procedure Example



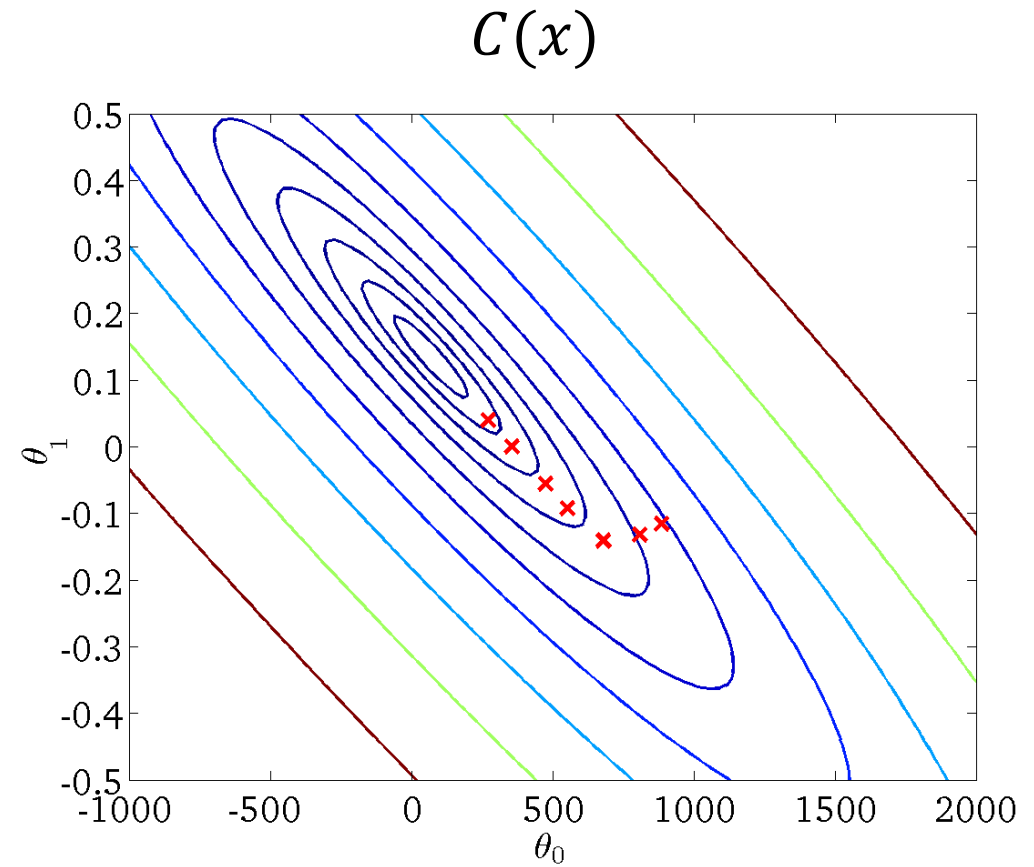
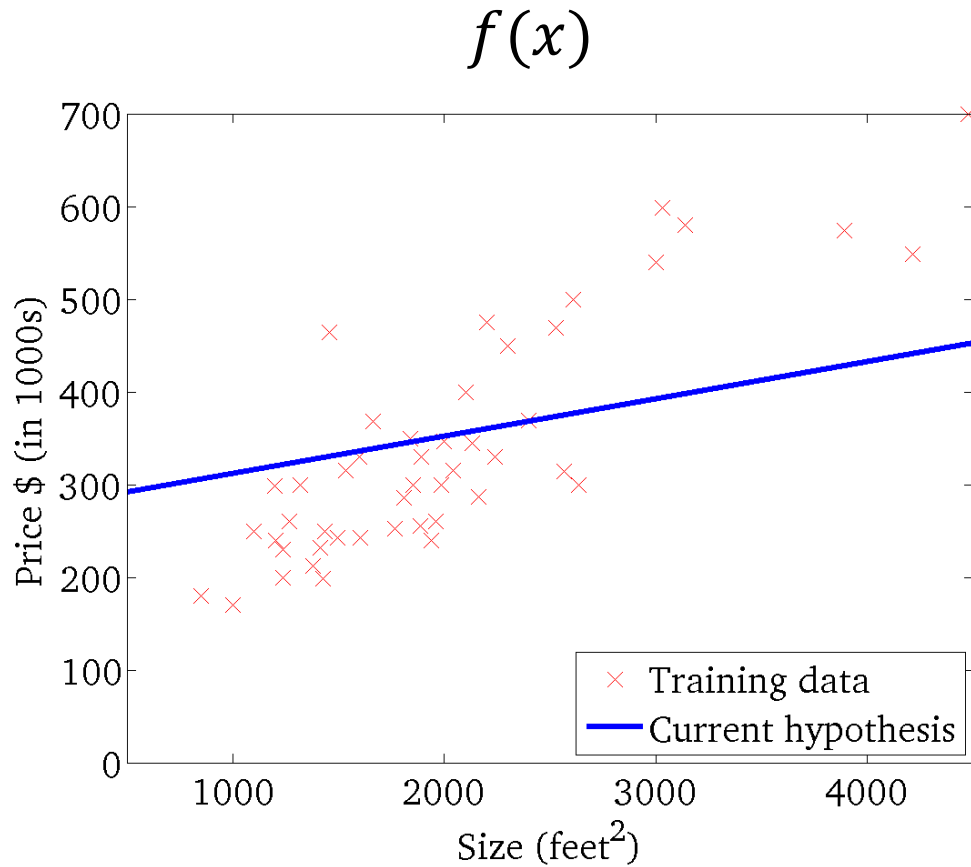
Training Procedure Example



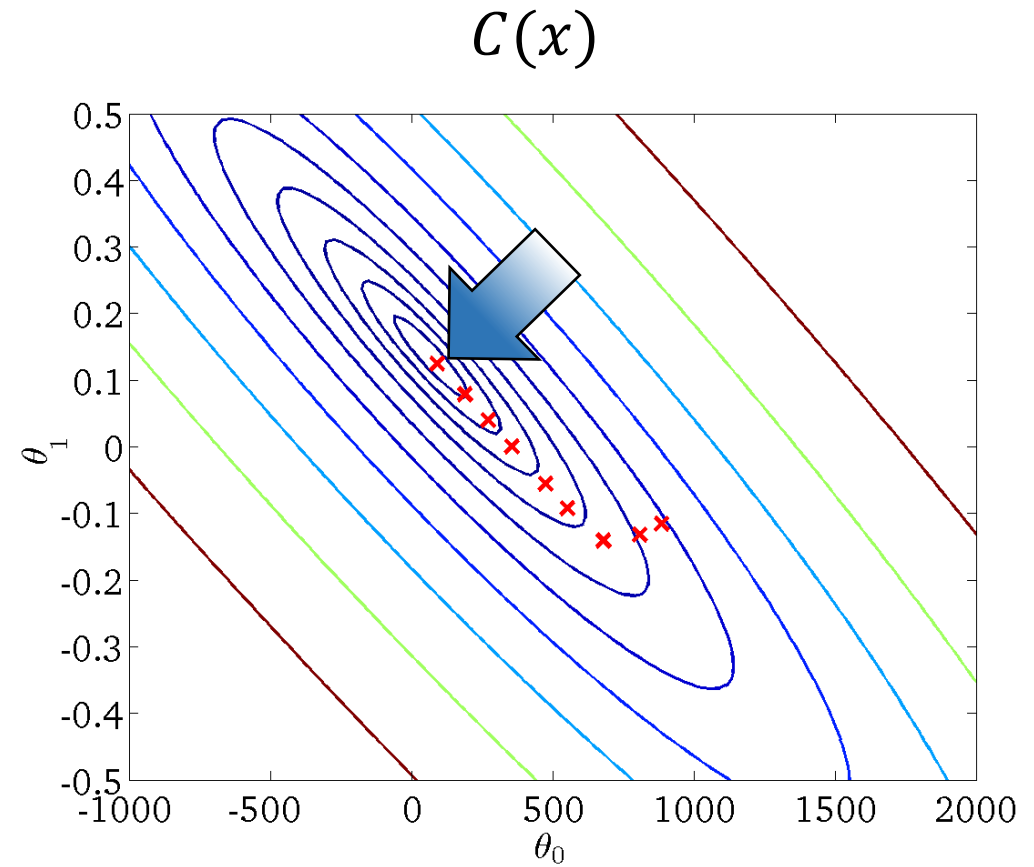
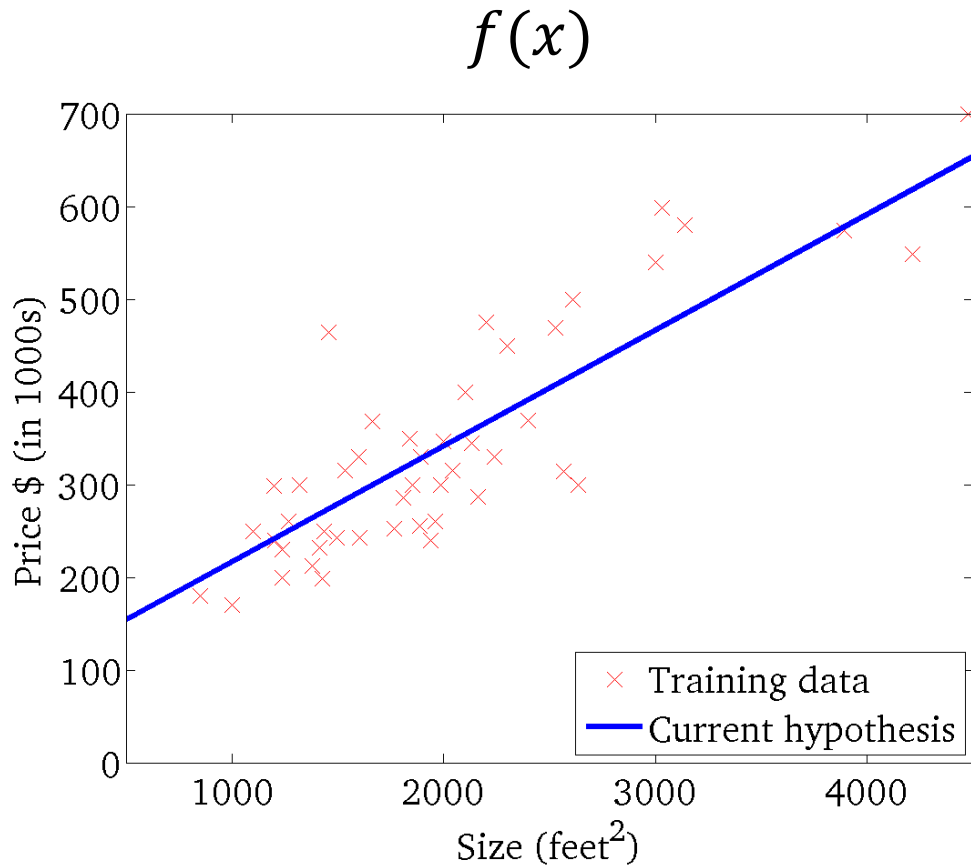
Training Procedure Example



Training Procedure Example



Training Procedure Example



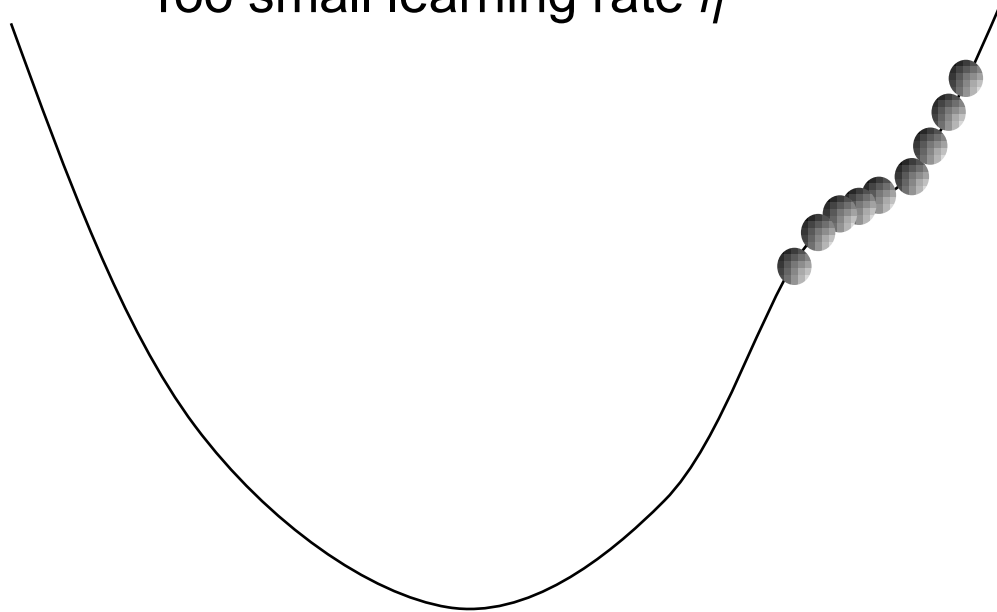
Learning Rate

↳ GD의 가장 중요한 파라미터

- The step size (learning rate) is one of the most critical components in the training.

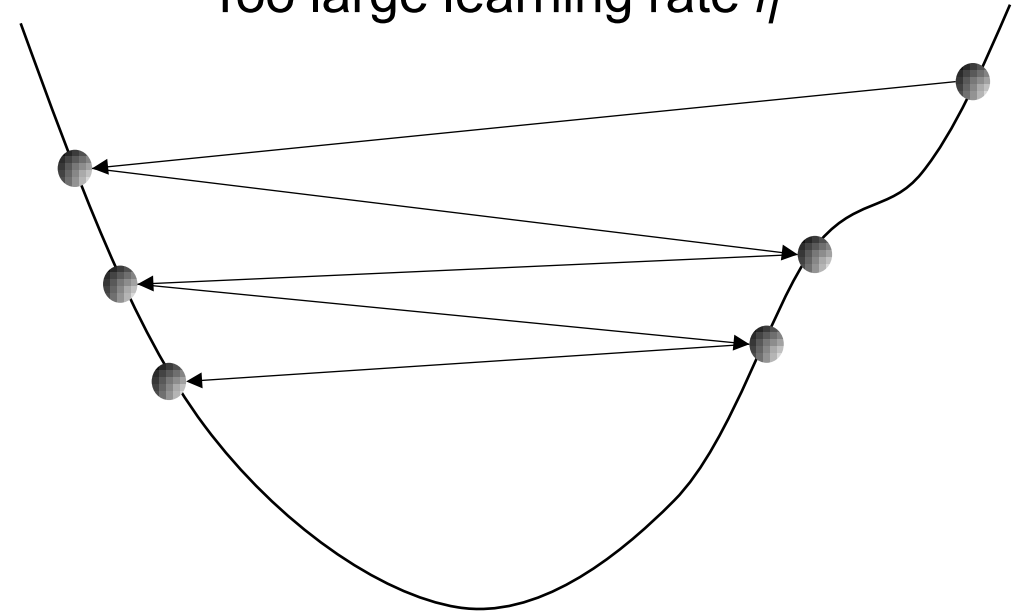
Cost Function이 발산할 수 있음
단, Convergence 가능하다면
좌측보다 효율적, 일반화 성능 ↑

Too small learning rate η



Training goes forever...

Too large learning rate η



May even diverge!

Hyper-Parameters

- A set of parameters that should be manually adjusted by users.
- Most of machine learning algorithms have hyper-parameters.
- Remember you have to 'play' with hyper-parameters until you achieve expected performance!
- We will be talking about this later again...

하이퍼 파라미터는 '학습' 시키지
않고 데이터에 따라 '튜닝' 함,
데이터에 상관없이 일반화하는게
불가능하기 때문



자주 사용되는 값으로 시작하되
빠르게 최적값으로 탐색하는 것이 중요

Closed Form Solution (1/2)

- Instead of GD, why not solve it analytically?

- Notice that the solution is when $\frac{dC(w)}{dw} = 0$.

- $$C(x) = \frac{1}{2n} (xw - y)^T (xw - y) = \frac{1}{2n} \sum_{i=1}^n (f(w, x^i) - y^i)^2$$
$$\propto w^T x^T xw - y^T xw - w^T x^T y + y^T y$$
$$= w^T x^T xw - 2w^T x^T y + y^T y$$

- Take derivative and set equal to 0, then solve for w :

$$\frac{d}{dw} (w^T x^T xw - 2w^T x^T y + y^T y) = 0$$
$$2x^T xw - 2x^T y = 0$$
$$x^T xw = x^T y$$

- Closed Form Solution: $w = (x^T x)^{-1} x^T y$

Closed Form Solution (2/2)

역행렬이 존재하면

- The solution can simply be obtained by plugging in x and y into

$$w = (x^T x)^{-1} x^T y$$

역행렬이 존재하지 않으면 (대부분의 경우)

- If $w = x^T x$ is not invertible (i.e., it's singular), you may want to
 - Use pseudo-inverse instead of the inverse.
 - In python, `numpy.linalg.pinv(a)`

↳ 정칙도가 희생되거나 계산은 빠른
정칙도 필요하면 GD로 대체해야함

Since the determinant is 0, the inverse matrix does not exist.

GD vs. Closed Form

Gradient Descent

- **Runs multiple iterations** (potentially many)
- **Need to find a good η**
- **Work well** when there are **many data points**.
- **Can support incremental learning**.

하이퍼 파라미터
튜닝 필요

시간복잡도 $\approx O(n)$

데이터 변화/
추가 시 대응 가능

Closed Form

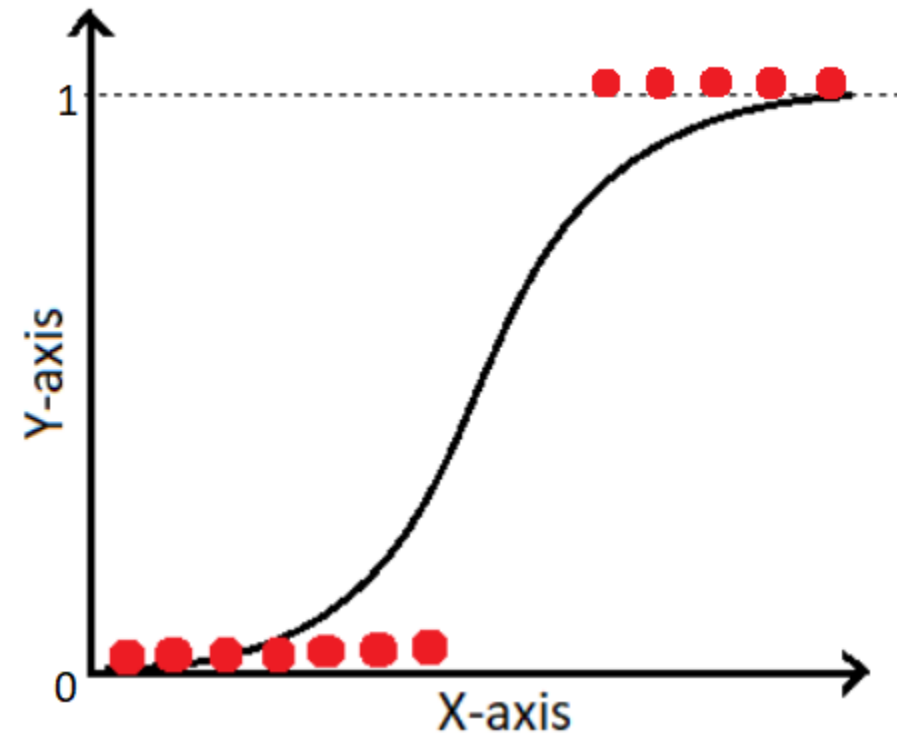
- **Non-iterative**
- **No need to find η**
- ***Slow*** when there are **many data points**.
- Remember the matrix inversion cost is roughly $O(n^3)$

하이퍼 파라미터
튜닝 필요 X

시간복잡도 매우 ↑

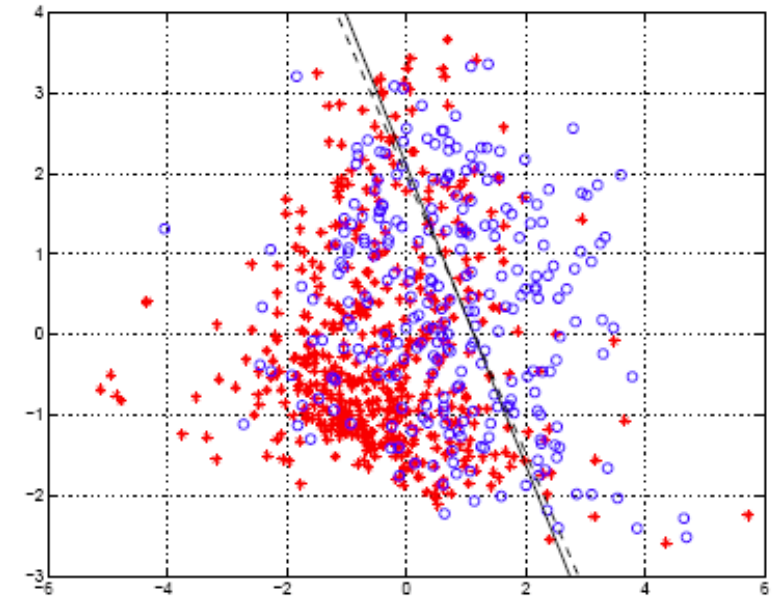
→ 기존 데이터와 유사한 분포의 데이터일 것이므로
기존 GD 과정을 반복하면 되기 때문에, Closed Form은 처음부터 다시 계산해야 함

- Supervised Learning?
- Linear Regression
- ***Logistic Regression***



Classification Based on Probability

- Instead of directly predicting the value, give the probability of the instance being one of pre-defined values (classes).
 - That is, learn $p(y|x)$.
- Assumptions
 - $0 \leq p(event) \leq 1$
 - $p(event) + p(\neg event) = 1$



* 특정 이벤트를 분류할때 분류 결과가 아니라 특정 값으로 분류될 '확률' 을 알 수 있음

What is Logistic Regression?

- Take a probabilistic approach to learning discriminative functions (i.e., a classifier).
 - We expect the model gives us $p(y = 1|x; w)$.
 - So, $0 \leq f(x) \leq 1$

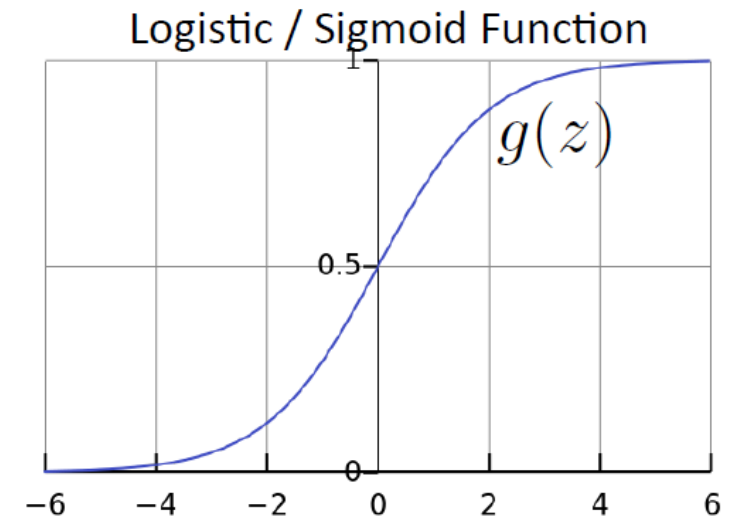
- Logistic Regression

- $f(x) = g(w^T x)$
- $g(x) = \frac{1}{1+e^{-z}}$ —————→ Sigmoid function

↓

$f(x) = \frac{1}{1+e^{-w^T x}}$

 linear regression + Sigmoid



Interpretation of Hypothesis Output

- $f(x)$ estimates $p(y = 1|x; w)$.

Example: Cancer diagnosis from tumor size

$$x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$$

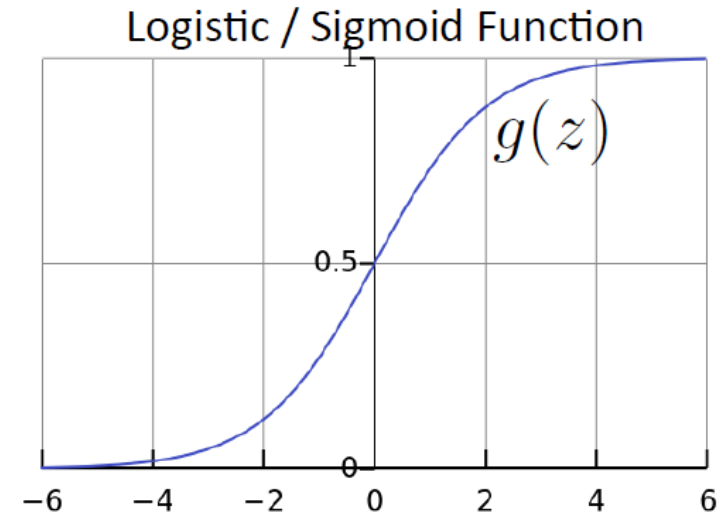
$$f(x) = 0.7$$

→ Tell patient that 70% chance of tumor being malignant.

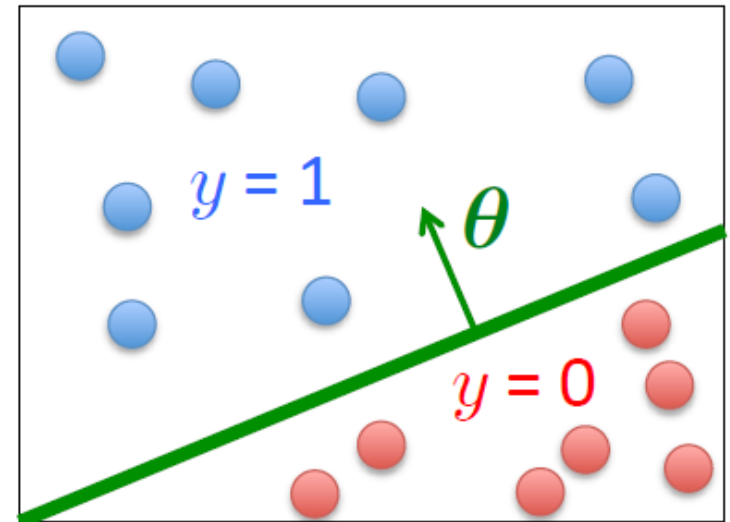
- Note that $p(y = 0|x; w) + p(y = 1|x; w) = 1$
- Therefore, $p(y = 0|x; w) = 1 - p(y = 1|x; w)$.

Logistic Regression

- $g(x) = \frac{1}{1+e^{-z}}$
- $w^T x$ should be a large negative for negative instances.
- $f(x) = \frac{1}{1+e^{-w^T x}}$
- $w^T x$ should be a large positive for positive instances.



- Assume the threshold is 0.5, then:
 - Predict $y = 1$ if $f(w^T x) \geq 0.5$
 - Predict $y = 0$ if $f(w^T x) < 0.5$



Cost Function in Logistic Regression

- Because our output is a probability, we cannot just use MSE.
- The model below forms a ***non-convex problem which is harder to solve than convex problems.***

$$f(x) = \frac{1}{1 + e^{-w^T x}}$$

- To address this issue, we will use a special cost function, which will be derived using Maximum Likelihood Estimation (MLE). 확신할 수 없음

입력에 상관없이 목표값에 대한 오류가
특정값 이하가 될 것을 확신할 수 있음

Aproximation

Maximum Likelihood Estimation (1/3)

- The likelihood of data is given by $l(w) = \prod_{i=1}^n p(y^i|x^i; w)$. 각각의 입력 x 에 대해 y 일 확률을 모두 곱한 것
- So, looking for the w that maximizes the likelihood,

$$w_{MLE} = \operatorname{argmax}_w l(w) = \operatorname{argmax}_w \prod_{i=1}^n p(y^i|x^i; w).$$

- We can take log without changing the solution as follows.

Because $\log(\cdot)$ is a monotonically increasing function, the solution of $f(x)$ also satisfies $\log(f(x))$.

$$\begin{aligned} w_{MLE} &= \operatorname{argmax}_w \log \prod_{i=1}^n p(y^i|x^i; w) \\ &= \operatorname{argmax}_w \sum_{i=1}^n \log(p(y^i|x^i; w)) \end{aligned}$$

Log-likelihood function
Log functions usually make
your life much easier...

Maximum Likelihood Estimation (2/3)

- ***Bernoulli distribution***: the discrete probability distribution of a random variable which takes the value 1 for the probability p and the value 0 for the probability of $1 - p$.
- If x is a random variable with Bernoulli distribution, then the expected value of x is as follows.
 - $\mathbb{E}[x] = p(x) \cdot x + (1 - p(x)) \cdot (1 - x)$
- Because x is either 0 or 1, we can also change the above equation to the below.
 - $\mathbb{E}[x] = p(x)^x (1 - p(x))^{1-x}$

Maximum Likelihood Estimation (3/3)

- Now, expand it as follows.

y가 Random Distribution 이라고 가정

$$\begin{aligned}w_{MLE} &= \underset{w}{\operatorname{argmax}} \sum_{i=1}^n \log(p(y^i|x^i;w)) && \text{Replace the likelihood with Bernoulli likelihood.} \\&= \underset{w}{\operatorname{argmax}} \sum_{i=1}^n \log(p(y^i=1|x^i;w)^{y^i} (1-p(y^i=1|x^i;w))^{1-y^i}) \\&= \underset{w}{\operatorname{argmax}} \sum_{i=1}^n \underbrace{\left[y^i \log(p(y^i=1|x^i;w)) + (1-y^i) \log(1-p(y^i=1|x^i;w)) \right]}_{\text{Probability mass function of a Bernoulli distribution}}\end{aligned}$$

- By substituting in model, we can have:

$$w_{MLE} = \underset{w}{\operatorname{argmin}} C(w),$$

The minus comes from
 $\operatorname{argmax} \rightarrow \operatorname{argmin}$

$$C(w) = - \sum_{i=1}^n [y^i \log f(x^i) + (1-y^i) \log(1-f(x^i))]$$

Intuition Behind the Cost Function (1/4)

$$C(w) = - \sum_{i=1}^n [y^i \log f(x^i) + (1 - y^i) \log(1 - f(x^i))]$$

- Cost of a single data point:

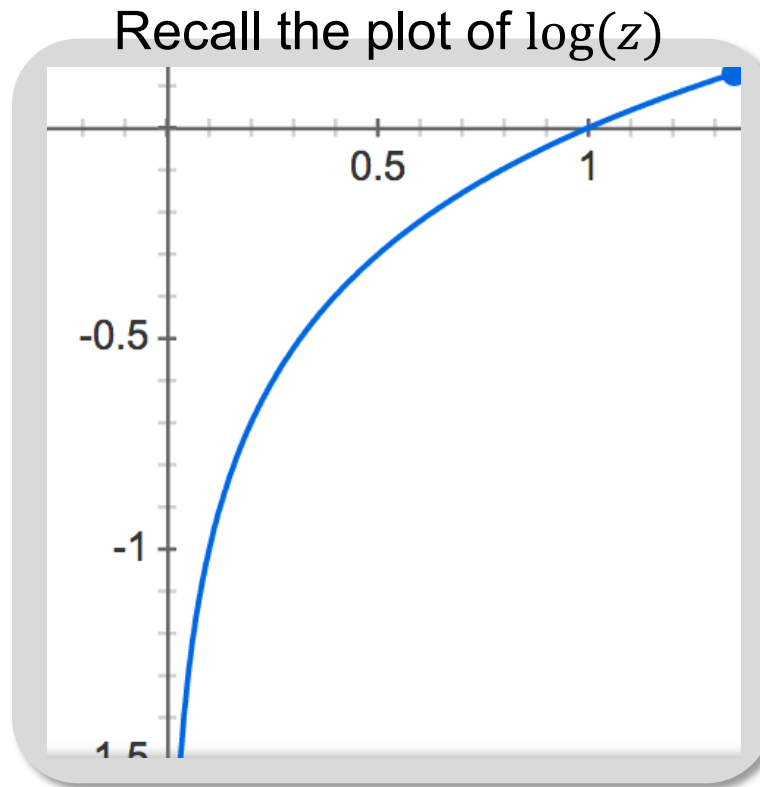
- $cost(f(x), y) = \begin{cases} -\log(f(x)), & \text{if } y = 1 \\ -\log(1 - f(x)), & \text{if } y = 0 \end{cases} \longrightarrow -\log(\text{Model Output})$

- Can re-write objective function to better understand its shape:

- $C(w) = \sum_{i=1}^n cost(f(x^i), y^i)$

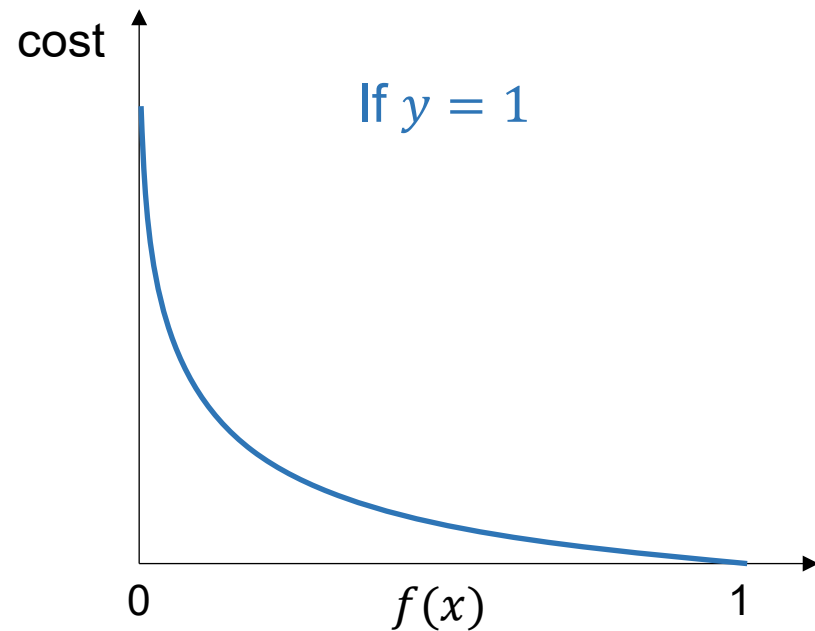
Intuition Behind the Cost Function (2/4)

- $cost(f(x), y) = \begin{cases} -\log(f(x)), & \text{if } y = 1 \\ -\log(1 - f(x)), & \text{if } y = 0 \end{cases}$



Intuition Behind the Cost Function (3/4)

$$\bullet \text{ cost}(f(x), y) = \begin{cases} -\log(f(x)), & \text{if } y = 1 \\ -\log(1 - f(x)), & \text{if } y = 0 \end{cases}$$

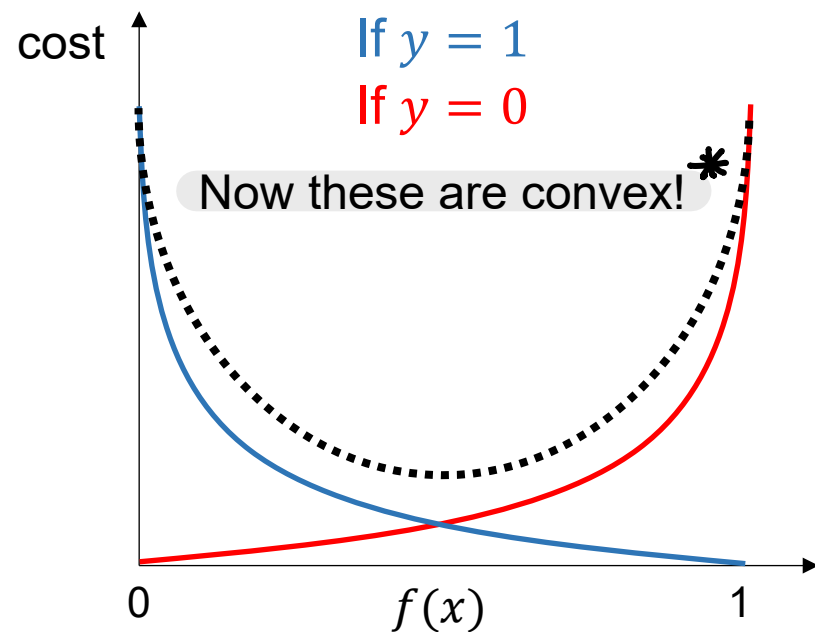


If $y = 1$,

- $\text{cost} = 0$ if prediction is correct.
- As $f(x) \rightarrow 0, \text{cost} \rightarrow \infty$.
- Capture intuition that larger mistakes should get larger penalties.
 - E.g., predict $f(x) = 0$, but $y = 1$.

Intuition Behind the Cost Function (4/4)

$$\bullet \text{ cost}(f(x), y) = \begin{cases} -\log(f(x)), & \text{if } y = 1 \\ -\log(1 - f(x)), & \text{if } y = 0 \end{cases}$$



If $y = 0$,

- $\text{cost} = 0$ if prediction is correct.
- As $1 - f(x) \rightarrow 0$, $\text{cost} \rightarrow \infty$.
- Capture intuition that larger mistakes get larger penalties.

1단계: 왜 MSE를 쓸 수 없는가?

로지스틱 회귀의 모델 출력은 시그모이드 함수 $f(x) = \frac{1}{1 + e^{-w^T x}}$ 입니다. 이 함수를 MSE의 형태로 비용 함수를 정의하면 $C(w) = \frac{1}{n} \sum (y^i - f(x^i))^2$ 가 되는데, 시그모이드 특성상 이 함수는 **non-convex**(비볼록)이 됩니다. Non-convex 문제는 local minimum이 여러 개 존재해 경사하강법이 global minimum을 보장하지 못하므로, 최적화가 어렵습니다. 따라서 **MLE**를 이용해 **convex**한 비용 함수를 새로 유도합니다.

첨부 파일

2단계: 우도함수(Likelihood) 설정

MLE의 핵심 아이디어는 “주어진 데이터를 가장 잘 설명하는 파라미터 w 를 찾자”입니다. 먼저 데이터 전체의 우도(likelihood) 를 정의합니다. 첨부 파일

$$l(w) = \prod_{i=1}^n p(y^i | x^i; w)$$

각 데이터 포인트 x^i 가 주어졌을 때 정답 y^i 가 나올 확률을 모두 곱한 것이 우도입니다. 이 곱이 성립하는 이유는 각 데이터 샘플이 서로 독립적(i.i.d.)이라 가정하기 때문입니다. 그러면 MLE 목표는 다음과 같습니다.

첨부 파일

3단계: 로그우도(Log-Likelihood)로 변환

곱셈($\prod$)은 수치적으로 다루기 매우 불편하고, 확률값(0~1 사이)을 계속 곱하면 값이 0에 수렴하는 **underflow** 문제가 생깁니다. 이를 해결하기 위해 단조증가 함수인 **log**를 씁니다. log는 단조증가 함수이므로, $l(w)$ 를 최대화하는 w 와 $\log l(w)$ 를 최대화하는 w 는 동일합니다. 첨부 파일

log의 성질 $\log(ab) = \log a + \log b$ 에 의해 곱이 합으로 바뀌어 훨씬 다루기 쉬워집니다.

4단계: 베르누이 분포 적용

로지스틱 회귀의 y 는 0 또는 1만 가질 수 있으므로, $p(y|x; w)$ 를 베르누이 분포로 모델링합니다. 첨부 파일
베르누이 분포의 확률질량함수(PMF)는 다음과 같습니다.

$$p(y|x; w) = p(y = 1|x; w)^y \cdot (1 - p(y = 1|x; w))^{1-y}$$

이 식이 왜 성립하는지 직관적으로 보면:

- $y = 1$ 일 때: $p^1 \cdot (1 - p)^0 = p \rightarrow 1$ 일 확률 그 자체
- $y = 0$ 일 때: $p^0 \cdot (1 - p)^1 = 1 - p \rightarrow 0$ 일 확률 그 자체

즉, 이 한 줄의 식이 두 경우를 모두 우아하게 표현합니다. 이를 로그우도에 대입하면: 첨부 파일

$$\sum_{i=1}^n [y^i \log p(y^i = 1|x^i; w) + (1 - y^i) \log(1 - p(y^i = 1|x^i; w))]$$

6단계: 비용함수의 직관적 의미

전체 합 $C(w)$ 를 샘플 하나의 비용으로 분리하면 다음과 같습니다. 첨부 파일 +1

이 각 경우를 그래프로 생각하면 이해가 쉽습니다. 첨부 파일

$y = 1$ 인 경우 ($-\log(f(x))$):

- $f(x) \rightarrow 1$ (모델이 1이라 올바르게 예측): $-\log(1) = 0 \rightarrow \text{cost} = 0$
- $f(x) \rightarrow 0$ (모델이 0이라 틀리게 예측): $-\log(0) \rightarrow \infty \rightarrow \text{무한대 페널티}$

$y = 0$ 인 경우 ($-\log(1 - f(x))$):

- $f(x) \rightarrow 0$ (모델이 0이라 올바르게 예측): $-\log(1) = 0 \rightarrow \text{cost} = 0$
- $f(x) \rightarrow 1$ (모델이 1이라 틀리게 예측): $-\log(0) \rightarrow \infty \rightarrow \text{무한대 페널티}$

이처럼 정답을 맞히면 페널티 0, 틀릴수록 페널티가 기하급수적으로 커지는 구조가 되며, 이 두 곡선 모두 볼록(**convex**) 형태를 갖기 때문에 경사하강법이 안정적으로 작동합니다. 첨부 파일 +1

Training (1/4)

- Let us use GD again to solve the minimization problem.
- A derivative of sigmoid function:

- $f(x) = \frac{1}{1+e^{-x}}$

- $$\begin{aligned} f(x)' &= \left((1 + e^{-x})^{-1} \right)' = -\left((1 + e^{-x})^{-2} \right) \frac{d}{dx} (1 + e^{-x}) \\ &= -\left((1 + e^{-x})^{-2} \right) \cdot (-e^{-x}) \\ &= \left((1 + e^{-x})^{-1} \right) \cdot \left((1 + e^{-x})^{-1} \cdot e^{-x} \right) \\ &= \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = f(x) \cdot (1 - f(x)) \end{aligned}$$

→ Sigmoid 함수의 경우 *
미분 매우 간단함

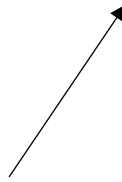
Training (2/4)

- Then, calculate the gradient of $C(w)$:

* $\log \Rightarrow \ln$

$$\begin{aligned} \bullet \frac{dC(w)}{dw} &= \frac{d \left(-\sum_{i=1}^n [y^i \log f(x^i) + (1-y^i) \log(1-f(x^i))] \right)}{dw} \\ &= -\sum_{i=1}^n \left(\frac{y^i}{f(x^i)} \cdot \frac{\partial f(x^i)}{\partial w} - \frac{(1-y^i)}{(1-f(x^i))} \cdot \frac{\partial f(x^i)}{\partial w} \right) \\ &= -\sum_{i=1}^n \left(\left(\frac{y^i}{f(x^i)} - \frac{(1-y^i)}{1-f(x^i)} \right) \cdot \frac{\partial f(x^i)}{\partial w} \right) \end{aligned}$$

$$\begin{aligned} f(x^i) &= \sigma(z^i), \quad z^i = w^\top x^i \\ \frac{\partial f(x^i)}{\partial w} &= f(x^i) (1 - f(x^i)) x^i \end{aligned}$$



Training (3/4)

$$\begin{aligned}\frac{dC(w)}{dw} &= - \sum_{i=1}^n \left(\left(\frac{y^i}{f(x^i)} - \frac{(1-y^i)}{1-f(x^i)} \right) \cdot f(x^i) (1-f(x^i)) x^i \right) \\ &= - \sum_{i=1}^n \left(y^i x^i (1-f(x^i)) - f(x^i) (1-y^i) x^i \right) \\ &= - \sum_{i=1}^n \left(y^i (1-f(x^i)) x^i - f(x^i) (1-y^i) x^i \right) \\ &= - \sum_{i=1}^n (f(x^i) - y^i) x^i\end{aligned}$$

→ Error의 정의가 동일하고 Convex Function을 사용하는 한 Cost Function의 Gradient는 동일

Training (4/4)

$$C(w) = - \sum_{i=1}^n [y^i \log f(x^i) + (1 - y^i) \log(1 - f(x^i))]$$

- $\frac{dC(w)}{dw} = \frac{1}{n} \sum_{i=1}^n [(f(x^i) - y^i)x^i]$

Long story short, this is the same as the MSE gradient! We will not go deep into how to derive this. If you are interested, please consult the textbook.

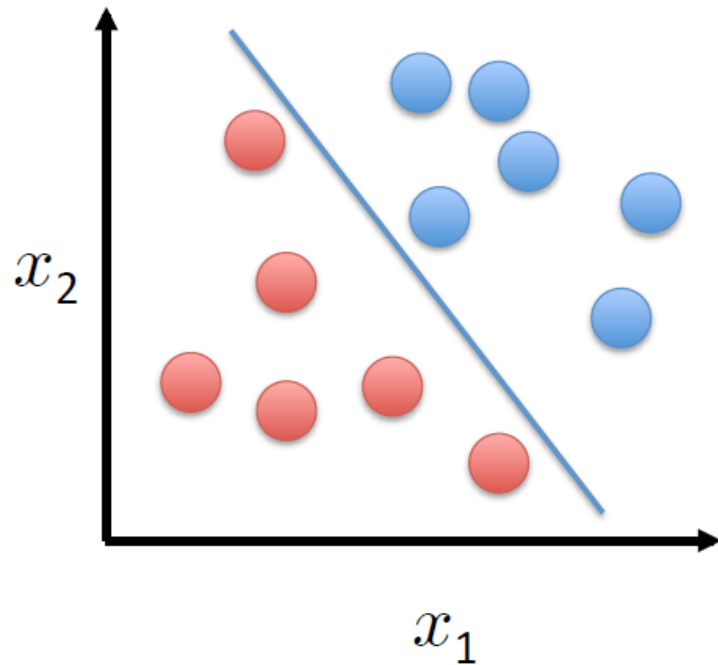
- Now, we can run GD as follows.

- $x_{t+1} = x_t - \eta \frac{dC}{dw}$

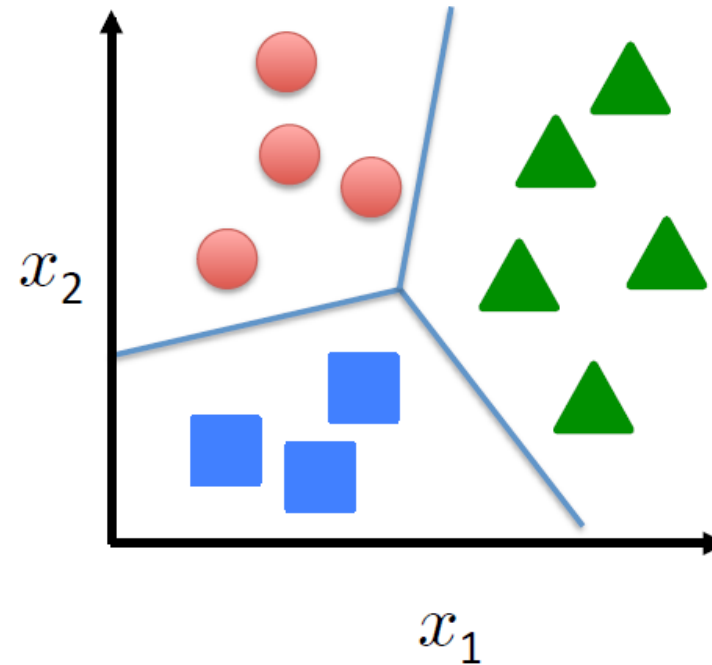
Multi-Class Classification

- What if the number of classes is larger than 2?

Binary Classification



Multi-Class Classification



Multi-Class Logistic Regression (1/3)

- For 2 classes:

- $f(x) = \frac{1}{1+\exp(-w^T x)} = \frac{\exp(w^T x)}{1+\exp(w^T x)}$

- For multi-classes $\{1, \dots, C\}$:

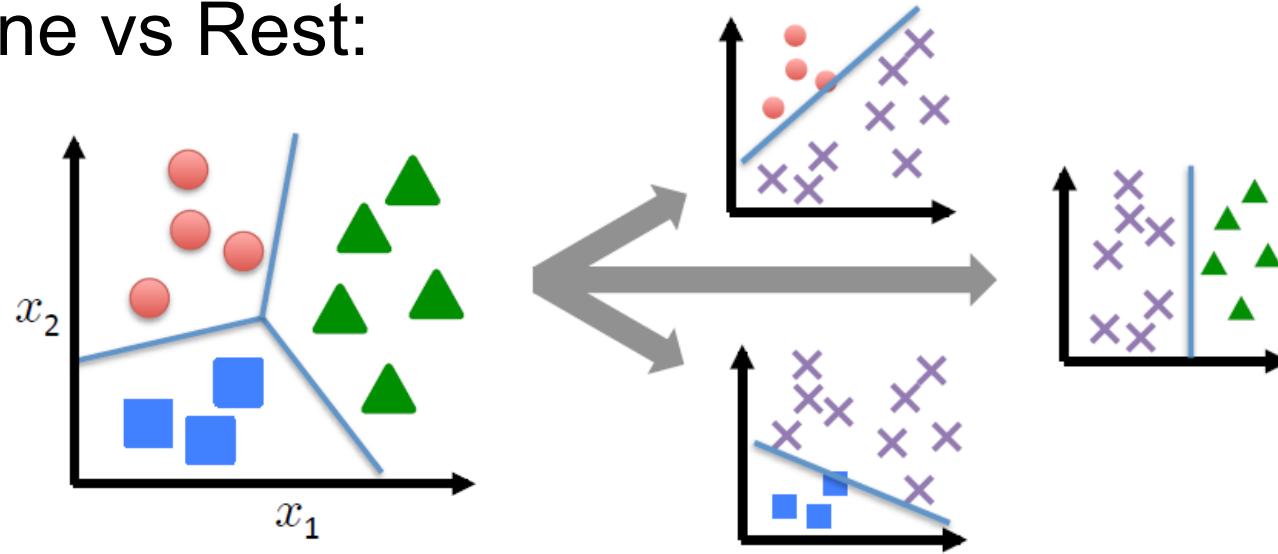
- $p(y = c|x; w_1, w_2, \dots, w_C) = \frac{\exp(w_c^T x)}{\sum_{c=1}^C \exp(w_c^T x)}$

This cost function is called 'softmax'.

Sigmoid for multi-classes ←
미분은 다르나 GD는 유사하게 사용가능

Multi-Class Logistic Regression (2/3)

- Split into One vs Rest:



- Train a logistic regression classifier for each class i to predict the probability that $y = i$ with:

- $$f_c(x) = \frac{\exp(w_c^T x)}{\sum_{c=1}^C \exp(w_c^T x)}$$

Multi-Class Logistic Regression (3/3)

- Use $f_c(x) = \frac{\exp(w_c^T x)}{\sum_{c=1}^C \exp(w_c^T x)}$ as the model for class c .
- Gradient Descent simultaneously updates all parameters for all classes.
 - The same gradients as binary class problems, just with above $f_c(x)$ for each output.
- Finally, predict the class label as the most probable one among C classes!

$$\max_c f_c(x)$$

Any Questions?

We now know how to use Machine Learning
to solve real-world problems!

